Platform Track

Interview Preparation Series • Professional Edition

Linux & the Command Line

Compiled with LaTeX • June 30, 2026

harsha.somisetty01@gmail.com

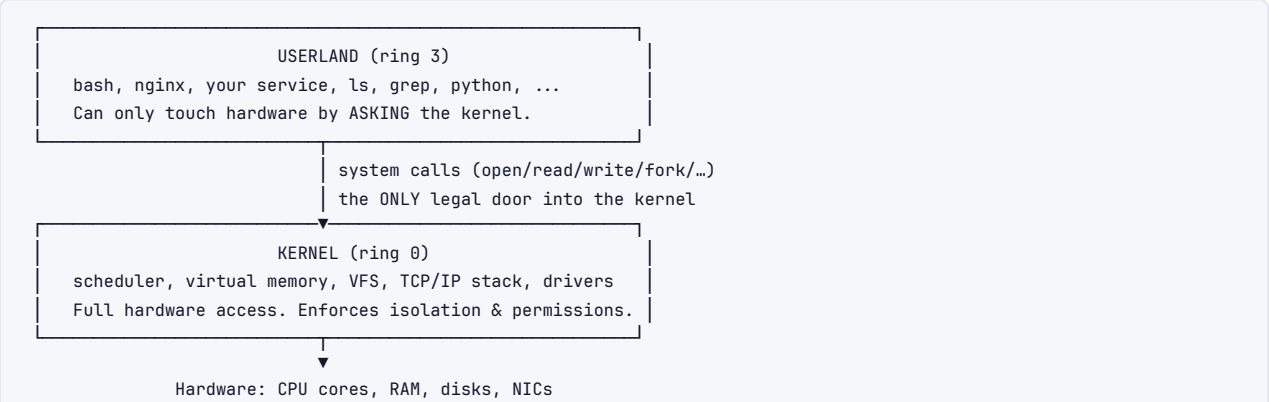
In This Edition

> **How to use this file:** Read top-to-bottom for deep, mechanism-level understanding. Jump to §Common Interview Questions + §Revision Cheat Sheet for last-minute revision. Linux fluency is *assumed* for backend/infra/SRE roles — "the server is slow, debug it" and "the disk is full" questions test it directly, and there is no faking it. Where this file references OS theory (scheduling, virtual memory, page cache) it points to §02 Operating Systems; where it references resource methodology it points to §09 Performance Engineering.

1 Overview --- What It Is

Linux is a Unix-like, open-source operating system **kernel**. Combined with GNU userland (coreutils, bash, glibc) it forms the OS that powers the overwhelming majority of servers, containers, cloud instances, Android phones, and embedded devices. When an interviewer says "deploy this service" or "debug this box," they mean a Linux box.

The system splits cleanly into two worlds:



The **shell** (bash, zsh, fish) is your text interface to this machine. Its philosophy, inherited from Unix in 1970:

> **Do one thing well. Make everything text. Compose programs with pipes.**

That is why a 10-character pipeline can answer a question that would take a custom program in most languages:

```bash
awk '{print $1}' access.log | sort | uniq -c | sort -rn | head
```

(top source IPs in a web access log — five small tools, one line).

Two ideas underpin everything else and you should be able to recite them:

1. **"Everything is a file."** Regular files, directories, devices (`/dev/sda`), pipes, sockets, and even kernel/process state (`/proc`, `/sys`) are exposed through the same file API (`open`, `read`, `write`, `close`). This is why `cat /proc/cpuinfo` works, why `echo 1 > /sys/...` tunes the kernel, and why redirection works uniformly across all of them.
2. **The kernel/userland split.** Your program runs unprivileged. It cannot touch the disk, the network card, or another process's memory directly — it must make a **system call**, which traps into the kernel (ring 0), does the privileged work, and returns. This boundary is what makes a multi-tenant server safe.

2 Why It Exists

Hardware is finite and shared; programs are many and untrusting. Without an OS kernel + a usable interface on top:

1. Every program would speak raw hardware (device registers, interrupt controllers).
2. One buggy program could corrupt another's memory or the whole machine.
3. There would be no way to *script* operations — no automation, no CI/CD, no reproducibility.

The **kernel** solves multiplexing, isolation, abstraction, and protection (the deep theory lives in §02 Operating Systems). The **CLI** exists on top because **text streams are the most composable, scriptable, automatable interface humans and machines have**. A GUI button does one fixed thing; a command can be:

- **Piped** into another command.
- **Redirected** to a file or another machine.
- **Looped** over thousands of inputs.
- **Committed to git** as a script, reviewed, and re-run identically a year later.

```
GUI:   click → one action, no record, not repeatable
CLI:   type → composable, loggable, scriptable, diff-able, automatable
```

Interview takeaway: The CLI is not "the old way" — it is the *programmable* way. SRE and backend work is automation, and automation is text in, text out. That is why fluency is non-negotiable for these roles.

3 Why FAANG Cares

- **Everything runs on Linux.** Google's Borg, Meta's fleet, AWS EC2/Lambda hosts, Netflix's Titus — all Linux. Operating a service *is* operating Linux.
- **On-call is a CLI exercise.** "Latency spiked at 14:00, the box is unhealthy — find out why" cannot be answered from a dashboard alone; eventually you SSH in and run `top`, `ss`, `dmesg`, `journalctl`, `strace`. Interviews simulate exactly this.
- **The disk-full / OOM / D-state class of outages is universal.** These have nothing to do with your application logic and everything to do with understanding inodes, the OOM killer, and process states. They separate people who have operated systems from people who have only written code.
- **Containers are Linux features.** Kubernetes resource limits = cgroups. Container isolation = namespaces. You cannot reason about a throttled or OOMKilled pod without knowing what those are.
- **Automation glue is bash.** Build scripts, deploy hooks, log rotation, cron jobs, entrypoints — all shell. A subtle `set -e` / quoting bug can take down a deploy.

Company-specific flavor of the same point:

Company	What they'll probe
Google / SRE	The USE method, load average meaning, `/proc`, structured triage, "non-abstract large system design" assumes you know what a process and a syscall cost.
Amazon (AWS)	EC2/Lambda hosts are Linux; "operational excellence" leadership principle → disk full, OOM, log triage, graceful shutdown.

Company	What they'll probe
Meta	Massive fleets; debugging at host level (`perf`, BPF), cgroup-based isolation, signal handling for clean restarts.
Netflix	Brendan Gregg literally works(ed) there — USE method, flame graphs, `bcc`/BPF tools originate from this culture.
Uber / Databricks / infra-heavy	Kernel tuning (`sysctl`, swappiness, ulimits), NUMA, epoll, JVM-on-Linux page-cache interactions.

Interview takeaway: When you say "I'd cache it in memory" or "the service restarts cleanly," the interviewer wants to know you understand page cache, SIGTERM handling, and cgroup limits underneath. Linux is where systems-design abstractions become real.

4 Core Concepts

Quick mental model of the primitives, each expanded in its own section below.

Concept	One-line definition	Where it bites you
Process	A running program: PID, address space, file descriptors, credentials.	Leaks, zombies, runaway CPU.
File descriptor (fd)	Small integer handle to an open file/socket/pipe. **0=stdin, 1=stdout, 2=stderr**.	"Too many open files", deleted-but-open files.
Inode	On-disk metadata record for a file (NOT its name).	`rm` is unlinking; disk full with no big files.
Permissions	rwx for owner/group/other, plus setuid/setgid/sticky.	`chmod 777`, privilege escalation.
Signal	Async kernel-delivered notification (`SIGTERM`, `SIGKILL`).	Graceful shutdown, hung processes.
Pipe / redirection	Wiring fds between processes and files.	Log triage, lost stderr.
Environment variable	`KEY=value` config inherited by children (`PATH`, `HOME`).	"command not found", config bugs.
Exit code	0 = success, 1–255 = failure (`$?`).	Script reliability, `&&`/`‖`
Namespace / cgroup	Kernel isolation + resource limits = containers.	OOMKilled / throttled pods.

4.1 The system call: the door into the kernel

When `bash` runs `cat file.txt`, an enormous amount happens, but the *load-bearing* events are system calls:

```
fork()    → clone bash into a child process
execve()  → child replaces itself with the `cat` program
open()    → cat asks kernel for a file descriptor to file.txt
read()    → cat asks kernel to copy bytes from disk into its buffer
write()   → cat asks kernel to copy bytes to fd 1 (stdout)
close()   → release the fd
exit()    → child terminates; bash wait()s to collect status
```

You can watch every one of these:

```bash
strace -f -e trace=open,openat,read,write,execve cat file.txt
```

Each syscall is a **mode switch** (user→kernel→user), costing ~100–1000 ns. This is why batching matters: one `write()` of 64 KB beats 64,000 `write()`s of 1 byte. (See §02 for the privilege-ring mechanism.)

Interview takeaway: A program can do nothing privileged on its own — every file, network, or memory-management action is a `syscall` that traps into the kernel. `strace` lets you see exactly which calls a process is making, which is gold for debugging hangs.

5 Filesystem & Inodes

5.1 The Filesystem Hierarchy Standard (FHS)

Linux puts things in predictable places. Know these cold — they come up constantly in debugging.

```
/                root of everything
├── bin   usr/bin essential + general user binaries (ls, cat, bash)
├── sbin sbin     system binaries (mount, ip, sysctl) — usually root
├── etc            CONFIGURATION (text). /etc/passwd, /etc/fstab, /etc/nginx/
├── var            VARIABLE data that changes at runtime:
│   ├── log        ← /var/log/syslog, nginx/, journal/  (logs fill disks!)
│   ├── lib        app state/databases
│   ├── cache      caches
│   └── spool      mail/print/cron queues
├── tmp            world-writable scratch (cleared on reboot, sticky bit)
├── home           user home directories
├── root           root user's home (NOT /)
├── proc           VIRTUAL fs: live kernel+process state (/proc/<pid>, /proc/meminfo)
├── sys            VIRTUAL fs: device & kernel tunables (/sys/...)
├── dev            device files (/dev/sda, /dev/null, /dev/urandom)
├── opt            optional/third-party software
├── mnt /media     mount points for external/temporary filesystems
└── lib usr/lib    shared libraries (libc.so, etc.)
```

`/proc` and `/sys` are **not on disk** — they are kernel data structures presented as files. `cat /proc/loadavg` reads live numbers; `echo 10 > /proc/sys/vm/swappiness` reconfigures the kernel. This is "everything is a file" taken to its logical extreme.

```bash
cat /proc/cpuinfo      # CPU model, cores, flags
cat /proc/meminfo      # memory breakdown
cat /proc/loadavg      # 1/5/15-min load + running/total procs
cat /proc/<pid>/status # per-process state, memory, threads
ls  /proc/<pid>/fd     # every file descriptor the process holds
cat /proc/mounts       # what's mounted where
```

5.2 Inodes --- what a file *really* is

A filename is **not** the file. The file is an **inode** (index node): a fixed-size on-disk record holding *all* the metadata and pointers to data blocks — but **not the name**.

```
Inode #1048593
┌──────────────────────────────────────┐
│ type:        regular file            │
│ permissions: rw-r--r--  (mode bits)  │
│ owner:       uid=1000  gid=1000      │
│ size:        4096 bytes              │
│ timestamps:  atime / mtime / ctime   │
│ LINK COUNT:  2   ← how many names point here
│ block pointers → [data block, data block, indirect…] │
└──────────────────────────────────────┘
        ▲               ▲
        │               │  (filename lives in the DIRECTORY, not here)
   directory entry   directory entry
   "report.txt"        "backup.txt"
```

A **directory** is just a special file mapping *names* → *inode numbers*:

```bash
ls -i                # show inode number next to each name
df -i                # show INODE usage per filesystem (separate from bytes!)
stat report.txt      # dump the full inode metadata
```

Why this matters in three concrete ways:

1. `rm` does not "delete" --- it *unlinks*

The syscall behind `rm` is `unlink()`. It removes one *name* → *inode* mapping and decrements the inode's **link count**. The data is only reclaimed when **both** of these reach zero:

- link count = 0 (no directory entry points to it), **and**
- open-fd count = 0 (no running process has it open).

2. A deleted-but-open file keeps its disk space (the classic gotcha)

```
nginx has /var/log/app.log open (fd 7) and is writing to it.
You run:  rm /var/log/app.log
  → unlink removes the NAME. link count → 0.
  → BUT nginx still holds fd 7. open count = 1.
  → The inode + its data blocks STAY ALLOCATED.

Result:  `df` still shows the disk 100% full.
         `du` and `ls` show NO large file — the name is gone!
```

This is the single most famous Linux debugging trap. You find it with `lsof`:

```bash
lsof | grep deleted          # processes holding deleted files
lsof -p <pid> | grep deleted
```

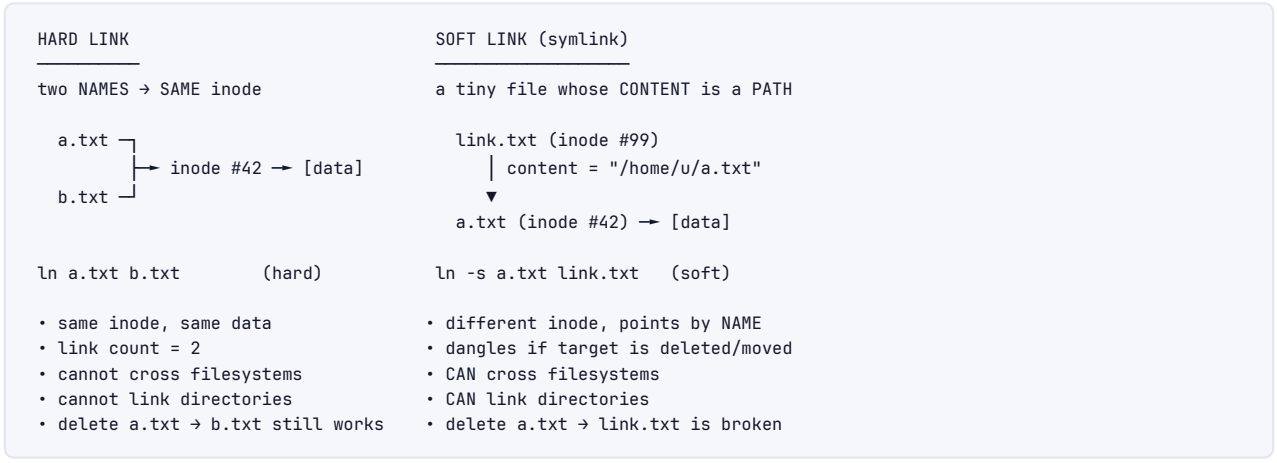
The fix is **not** another `rm` (the name is already gone) — you must make the process release the fd: truncate it in place, signal the process to reopen its log (`logrotate` sends SIGHUP), or restart it.

```bash
: > /proc/<pid>/fd/7         # truncate the still-open file via /proc — space freed instantly
```

3. Disk full but no big files = out of *inodes*

A filesystem has a fixed number of inodes set at creation. Millions of tiny files (session files, cache fragments, mail) can exhaust inodes while bytes are nearly empty. `df -h` says there's space; writes still fail with `No space left on device`. `df -i` reveals the truth.

5.3 Hard links vs soft (symbolic) links



Bash

```
ln  target hardlink          # hard link (rare, same fs)
ln -s target symlink         # symbolic link (common: /usr/bin/python → python3)
ls -l                        # symlinks show  link → target ;  hard links look like normal files
readlink -f symlink          # resolve to final real path
```

Interview takeaway: A filename is a pointer to an inode, not the file itself. `rm` decrements a link count; data dies only when no name AND no open fd remain. That is why a deleted-but-open log file keeps the disk full — and why `lsof | grep deleted` is the cure.

5.4 Mounting, `df` vs `du`

A **mount** grafts a filesystem (on a device, a network share, or a virtual fs) onto a directory in the single unified tree. There are no drive letters; `/`, `/home`, `/var` may be separate disks.

Bash

```
mount                        # list everything mounted
mount /dev/sdb1 /mnt/data    # attach a device at /mnt/data
findmnt                      # tree view of mounts
cat /etc/fstab               # mounts applied at boot
```

`df` and `du` answer different questions and often disagree — knowing why is an interview favorite:

	`df` (disk free)	`du` (disk usage)
Source	Filesystem superblock counters	Walks the directory tree, sums file sizes
Speed	Instant	Slow on big trees
Sees deleted-but-open files?	**Yes** (space still allocated)	**No** (no directory entry to walk)
Sees files under a mount point	Counts the underlying fs	Stops at mount boundary (`du -x`)

	`df` (disk free)	`du` (disk usage)
Use for	"Is the disk full?"	"*What* is using the space?"

```bash
df -h                       # human-readable free space per filesystem
df -i                       # INODE usage (the "full but no big files" case)
du -sh /var/* | sort -rh    # biggest directories under /var, largest first
du -xh --max-depth=1 / | sort -rh | head   # top space hogs, staying on one fs
```

When `df` says 100% but `du -sh /*` adds up to far less → suspect a **deleted-but-open file** (use `lsof | grep deleted`) or a large file **hidden under a mount point** (something mounted over a directory that already had data).

6 Permissions & Ownership

6.1 The rwx model

Every file/directory has an **owner (user)**, a **group**, and a mode for three classes: owner, group, others. Each class gets read (r), write (w), execute (x).

```
$ ls -l report.sh
-rwxr-xr--  1  alice  devs  812  Jun 17 10:00  report.sh
 ||||||||||     |      |
 |||||||||      |      └ group  = devs
 ||||||||       └──────── owner  = alice
 |||||||└ others: r--   (read only)
 ||||||└─ group:  r-x   (read + execute)
 |||||└── owner:  rwx   (read + write + execute)
 ||||└─── type:   - file | d dir | l symlink | c/b device | s socket | p fifo
```

What rwx means for files vs directories is different — a classic gotcha:

Bit	On a **file**	On a **directory**
`r`	read contents	**list** names inside (`ls`)
`w`	modify contents	**create/delete/rename** entries inside
`x`	execute as program	**enter/traverse** (`cd`, access files within)

Consequence: you can have `r` on a directory but not `x` → you can see the names (`ls`) but not access the files. You can have `x` but not `r` → you can `cd` in and open a file *if you know its exact name*, but `ls` fails. And `w` on a directory lets you **delete a file you don't own** (deletion is a directory operation, not a file operation) — the sticky bit exists to stop exactly this.

6.2 Octal notation

`r=4, w=2, x=1`. Sum per class, three digits for owner/group/other.

```python
rwx = 4+2+1 = 7      r-x = 4+0+1 = 5      r-- = 4 = 4
rw- = 4+2   = 6      r-- = 4     = 4      --- = 0

755 = rwxr-xr-x   directories, executables (everyone can enter/run, only owner writes)
644 = rw-r--r--   regular files (owner writes, world reads)
```

```
600 = rw-------   secrets: SSH private keys, ~/.aws/credentials (owner-only)
700 = rwx------   private directories
640 = rw-r-----   readable by owner + group, e.g. a config a service reads
```

```bash
chmod 644 file              # set absolute mode (octal)
chmod u+x script.sh         # symbolic: add execute for user
chmod go-w file             # remove write for group+other
chmod -R 755 dir/           # recursive (careful!)
chown alice:devs file       # change owner AND group
chown -R www-data: /var/www # recursive owner change, keep group
chgrp devs file             # change group only
```

> `ssh` will **refuse** to use a private key that is group/world-readable (`Permissions 0644 ... are too open`). The fix is `chmod 600 ~/.ssh/id_ed25519`. This is a real, frequent papercut.

6.3 Special bits: setuid, setgid, sticky

Beyond rwx there are three special bits, shown in the execute position:

```
-rwsr-xr-x   setuid  (s in OWNER  x slot): run as the FILE'S OWNER, not the caller
-rwxr-sr-x   setgid  (s in GROUP  x slot): run as file's group / dir: new files inherit dir's group
drwxrwxrwt   sticky  (t in OTHER  x slot): only the OWNER of a file may delete it (dirs)
```

setuid --- controlled privilege escalation

The canonical example is `/usr/bin/passwd`:

```bash
$ ls -l /usr/bin/passwd
-rwsr-xr-x 1 root root 68208 ... /usr/bin/passwd
   ^ the 's'
```

A normal user must update `/etc/shadow` (root-owned, `600`) to change their password. They obviously can't write to `/etc/shadow` directly. `passwd` is **owned by root with the setuid bit**, so when *any* user runs it, the process runs with root's effective UID for the duration — allowing the controlled, narrow operation of updating the password. setuid is power; a setuid-root binary with a bug is a privilege-escalation hole, which is why they are audited heavily.

setgid --- shared collaboration directories

On a **directory**, setgid makes every new file inside inherit the *directory's* group rather than the creator's primary group — so a team sharing `/srv/project` (group `devs`, setgid set) all produce files owned by `devs`, keeping the directory collaborative.

sticky bit --- the `/tmp` problem

`/tmp` is world-writable (`777`) so any user can create temp files. But `w` on a directory normally lets you delete *anyone's* files. Without protection, user A could delete user B's temp files. The **sticky bit** (`t`) restricts deletion to the file's owner:

```bash
$ ls -ld /tmp
drwxrwxrwt 10 root root ... /tmp
         ^ sticky bit: world-writable, but you can only delete YOUR OWN files
```

```bash
chmod u+s binary    # setuid
chmod g+s dir       # setgid
chmod +t dir        # sticky bit
chmod 4755 binary   # leading 4 = setuid (2 = setgid, 1 = sticky)
chmod 1777 dir      # leading 1 = sticky (like /tmp)
```

6.4 umask --- the default-permission mask

New files don't get `777`/`666`; the **umask** *subtracts* permission bits at creation. Default umask `022` means "remove write for group and other":

```python
file base 666  - umask 022  = 644  (rw-r--r--)
dir  base 777  - umask 022  = 755  (rwxr-xr-x)
```

A stricter `umask 077` makes everything owner-only by default (good for sensitive servers). `umask` is set in shell profiles / PAM and inherited by services.

6.5 ACLs --- when rwx isn't enough

The owner/group/other model can't express "user bob and user carol both get write, but no one else." **POSIX ACLs** add per-user/per-group entries:

```bash
setfacl -m u:bob:rw  report.txt    # grant bob read+write
getfacl report.txt                 # view all ACL entries
# ls -l shows a trailing '+' :  -rw-rw-r--+
```

6.6 Least privilege & why `chmod 777` is dangerous

`chmod 777` grants **everyone** read, write, and execute. On a web-served directory it means any user (or any compromised process, or an attacker who got a foothold) can overwrite your files, drop a malicious script, and execute it. It's the classic "it works now!" fix that opens a barn door.

```
chmod 777 /var/www      ×  anyone can plant & run code
                        ✓  chown www-data:www-data /var/www && chmod 755
                           (give the SERVICE ownership, grant the minimum)
```

The principle is **least privilege**: grant the *smallest* set of permissions that makes the task work. Run services as dedicated unprivileged users (`www-data`, `postgres`), `600` your secrets, avoid `sudo` for routine work, and never reach for `777`.

Interview takeaway: Octal `rwx` (755/644/600) plus three special bits: setuid (run as owner — `passwd`), setgid (inherit dir group — shared folders), sticky (`/tmp`: only delete your own). `chmod 777` is a security hole; the fix is correct *ownership* plus least-privilege modes, not blanket permissions.

7 Processes

7.1 What a process is

A **process** is an instance of a running program with: a **PID**, a **PPID** (parent's PID), its own **virtual address space**, a table of **open file descriptors**, **credentials** (UID/GID), **environment variables**, signal dispositions, and a current working directory. (Address-space internals — stack/heap/text, page tables — are in §02.)

7.2 fork / exec / wait --- how processes are born

Unix creates processes with a deliberately split pair:

```
fork()  — duplicate the current process (copy-on-write). Returns:
            • child:  0
            • parent: child's PID
            • -1:     error

execve() — REPLACE the current process image with a new program.
            Same PID, but code/heap/stack are wiped and reloaded.

wait()  — parent blocks until child exits, COLLECTS its exit status
            (this is "reaping" — prevents zombies).
```

A shell running `ls` does exactly this:

```c
pid_t pid = fork();          // clone the shell
if (pid == 0) {              // in the child:
    execve("/bin/ls", ...);  // become `ls`
} else {                     // in the parent (the shell):
    waitpid(pid, &status, 0);// wait for ls, read its exit code → $?
}
```

`fork()` is cheap because of **copy-on-write**: parent and child share physical pages marked read-only; a page is only copied when one side writes it. So `fork()` immediately followed by `execve()` (which throws the address space away anyway) barely copies anything. (See §02 for CoW mechanics.)

7.3 PID, PPID, and the process tree

Every process except `init`/`systemd` (PID 1) has a parent. The tree is observable:

```bash
pstree -p              # visualize the process tree
ps -ef                 # full listing with PPID column
ps -o pid,ppid,stat,comm -p <pid>
```

PID 1 (`systemd` or `init`) is special: it is the ancestor of everything and the **reaper of orphans** (below).

7.4 Process states: R, S, D, Z, T

`ps`/`top` show a one-letter state. Knowing what each means — especially **D** — is a senior signal.

State	Name	Meaning	Killable?
R	Running / Runnable	On CPU now, or in the run queue ready to go.	yes
S	Interruptible sleep	Waiting for an event (I/O, timer, lock); *can* be woken by a signal. **Most idle processes.**	yes
D	**Uninterruptible sleep**	Blocked in the kernel on I/O that can't be interrupted (e.g. disk, NFS). **Does NOT respond to signals — not even SIGKILL.**	**NO**
Z	Zombie	Exited, but parent hasn't `wait()`ed. Holds only a PID-table slot.	already dead
T	Stopped	Suspended by SIGSTOP / Ctrl-Z, or under a debugger.	resume w/ SIGCONT

```
R  ──────► running on a core (or runnable, waiting for a core)
S  ──────► "I'm waiting for the network / a timer" (normal, healthy)
D  ──────► "I'm stuck in the kernel waiting for disk/NFS" (DANGER if many)
Z  ──────► "I'm done but nobody collected my exit code"
T  ──────► "I've been paused"
```

The D-state lesson (huge in interviews): a process in `D` is in **uninterruptible sleep** — usually blocked on a disk or a hung NFS mount. `kill -9` **will not work**, because the process isn't running any code that could receive the signal; it's parked inside a kernel call. A pile of D-state processes is the classic fingerprint of an **I/O problem** (a dying disk, a stalled storage backend, an NFS server that went away). The fix is to address the I/O, not to kill harder.

Bash

```bash
ps -eo pid,state,wchan:25,comm | awk '$2=="D"'   # list D-state procs + what kernel fn they're stuck in
```

7.5 Zombies vs orphans --- and how to avoid them

These two are constantly confused; nail the distinction:

```
ZOMBIE (defunct)                    ORPHAN
────────────────                    ──────
child EXITED, parent ALIVE          parent DIED, child ALIVE
but parent never wait()ed           child gets RE-PARENTED to PID 1
                                    (systemd), which WILL wait() it

  parent (busy, no wait)              parent × (gone)
      │ (ignores)                         ┆ (broken link)
      ▼                                   ▼
  child <defunct>  ← stuck            child → adopted by PID 1 → reaped cleanly
```

- A **zombie** consumes only a process-table slot, but thousands of them can exhaust the PID space. They're a **bug in the parent** (it isn't reaping). Shows as `Z` / `<defunct>` in `ps`.
- An **orphan** is *not* a problem: PID 1 adopts it and reaps it when it exits.
- **How to avoid zombies:** the parent must `wait()`/`waitpid()` on children — typically by handling `SIGCHLD`, or in containers by running a proper init/`tini` as PID 1 (a naive PID-1 process that doesn't reap leaves zombies piling up — a real Docker footgun).

Bash

```bash
ps aux | awk '$8 ~ /Z/'     # list zombies (STAT column contains Z)
# You cannot kill a zombie (it's already dead). You fix/kill/signal its PARENT to reap it.
```

7.6 Inspecting and prioritizing processes

```bash
ps aux                          # BSD-style: all processes, user-oriented columns
ps aux --sort=-%cpu | head      # top CPU consumers
ps aux --sort=-%mem | head      # top memory consumers
ps -ef                          # System-V style: shows PPID
pgrep -fl nginx                 # find PIDs by name/cmdline
pidof nginx                     # PIDs of a program
top                             # live, interactive (press M=mem, P=cpu, 1=per-core)
htop                            # nicer top: tree view, scroll, kill by F9
```

Niceness & scheduling priority. The `nice` value ranges from `-20` (highest priority, hoggish) to `+19` (lowest, "nicest"). It biases the CFS scheduler's CPU share (see §02 CFS):

```bash
nice -n 10 ./batch_job          # start a job at lower priority (+10)
renice -n 5 -p <pid>            # change priority of a running process
renice -n -5 -p <pid>           # raise priority (needs root for negatives)
ionice -c3 -p <pid>             # I/O priority: class 3 = idle (only when disk is free)
```

Use `nice`/`ionice` to keep a heavy batch job (backup, reindex) from starving the latency-sensitive service sharing the box.

7.7 Process groups & sessions (why Ctrl-C hits the whole pipeline)

Processes are organized into **process groups** (a pipeline is one group) and **sessions** (a login/terminal). The terminal has a **foreground process group**; keyboard signals (Ctrl-C → SIGINT, Ctrl-Z → SIGSTOP) go to *all* members of that group. That is why `Ctrl-C` on `grep x huge.log | sort | uniq` kills the whole pipeline, not just one stage. A **session leader** owns the controlling terminal; daemons deliberately `setsid()` to detach from any terminal so they survive logout.

Interview takeaway: `fork+exec+wait` is how Unix spawns programs; CoW makes fork cheap. Memorize the states — especially `D` (uninterruptible sleep = stuck on I/O, immune to `kill -9`, a sign of disk/NFS trouble) and `Z` (zombie = parent failed to reap). Zombies are a parent bug; orphans are harmless (PID 1 adopts them).

8 Signals & Job Control

8.1 What signals are

A **signal** is an asynchronous notification delivered by the kernel to a process. It carries *only a number* (no payload). The process can, for most signals, install a **handler**, **ignore** it, or take the **default action** (often "terminate"). Two signals can never be caught, blocked, or ignored: `SIGKILL` and `SIGSTOP`.

8.2 The signal table you must know

Signal	#	Default action	Catchable?	Typical use
`SIGHUP`	1	terminate		terminal closed; by convention "**reload config**" for daemons
`SIGINT`	2	terminate		**Ctrl-C** — interrupt foreground job

Signal	#	Default action	Catchable?	Typical use
`SIGQUIT`	3	term + core dump		Ctrl-\
`SIGKILL`	9	terminate	****	**force kill — uncatchable**, last resort
`SIGSEGV`	11	term + core		invalid memory access (segfault)
`SIGPIPE`	13	terminate		wrote to a pipe with no reader (e.g. '...
`SIGTERM`	15	terminate		**polite shutdown — the default `kill`**
`SIGSTOP`	19	stop	****	pause (uncatchable), like Ctrl-Z's underlying stop
`SIGCONT`	18	continue		resume a stopped process
`SIGCHLD`	17	ignore		sent to parent when a child stops/exits (reaping hook)
`SIGUSR1/2`	10/12	terminate		app-defined (nginx uses USR1 to reopen logs)

Bash

```bash
kill <pid>              # sends SIGTERM (15) by default — graceful
kill -TERM <pid>        # explicit graceful
kill -9 <pid>           # SIGKILL — forceful, no cleanup
kill -HUP <pid>         # ask a daemon to reload config
kill -l                 # list all signal names/numbers
pkill -f "python app"   # kill by command-line pattern
killall nginx           # kill all processes named nginx
kill -STOP <pid> ; kill -CONT <pid>   # pause then resume
```

8.3 SIGTERM vs SIGKILL --- the graceful-shutdown story

This is *the* signals interview question.

```
SIGTERM (15)                          SIGKILL (9)
────────────                          ───────────
"Please shut down."                   "Die NOW."
Process CAN catch it →                Kernel destroys the process.
  flush buffers                       No handler runs.
  finish in-flight requests           No cleanup. No flush.
  close DB connections                Possible data loss / corruption.
  deregister from load balancer       Use ONLY when TERM is ignored
  exit(0)                             or the process is wedged.
```

The correct shutdown sequence (what `systemctl stop`, Kubernetes pod termination, and `docker stop` all do):

```
1. Send SIGTERM.
2. Wait a grace period (e.g. 30s; K8s terminationGracePeriodSeconds).
3. If still alive → send SIGKILL.
```

A well-behaved service **traps SIGTERM**, stops accepting new work, drains in-flight requests, and exits cleanly — enabling **zero-downtime deploys**. A service that ignores SIGTERM gets SIGKILLed mid-request → dropped connections, half-written data.

8.4 Catching signals in shell: `trap`

```bash
#!/usr/bin/env bash
cleanup() {
  echo "caught signal, cleaning up..."
  rm -f /tmp/work.$$           # remove temp files
  kill "$child_pid" 2>/dev/null
  exit 0
}
trap cleanup SIGTERM SIGINT    # run cleanup() on TERM or Ctrl-C
trap 'echo "done"' EXIT        # EXIT pseudo-signal: always runs on script exit

long_running_thing &
child_pid=$!
wait "$child_pid"
```

`trap` is how scripts and services implement graceful shutdown and guaranteed cleanup. You **cannot** trap `SIGKILL` or `SIGSTOP` — that's the whole point of them.

8.5 Graceful shutdown pattern (real services)

```
SIGTERM received
   |
   ├─► stop accepting NEW connections (close listen socket / fail readiness probe)
   ├─► let load balancer notice & route away (drain window)
   ├─► finish IN-FLIGHT requests (bounded by a deadline)
   ├─► flush buffers, commit, close DB/queue connections
   └─► exit(0)
   (if not done within grace period → orchestrator sends SIGKILL)
```

8.6 Job control (interactive shell)

```bash
sleep 1000 &        # run in background; shell prints [1] 12345 (job#, PID)
jobs                # list this shell's jobs
fg %1               # bring job 1 to foreground
bg %1               # resume a stopped job in the background
Ctrl-Z              # SIGTSTP: suspend the foreground job (state T)
Ctrl-C              # SIGINT: interrupt the foreground job
disown %1           # detach job from shell (survives shell exit)
nohup ./svc &       # ignore SIGHUP so the job survives terminal close; logs → nohup.out
```

`nohup`, `disown`, and `setsid` all solve "keep running after I log out." For real services you'd use `systemd` (next section) rather than `nohup`, but on a quick remote box `nohup ./script &` is the pragmatic move. `tmux`/`screen` are the heavier, more robust answer (persistent sessions you can re-attach to).

Interview takeaway: SIGTERM is a catchable "please stop" that lets a process flush and drain — the foundation of zero-downtime deploys; SIGKILL (`-9`) is uncatchable and skips all cleanup. Always TERM first, KILL only if it won't die. `trap` implements graceful shutdown in scripts; you can never trap KILL or STOP.

9 Pipes, Redirection & File Descriptors

9.1 The three standard file descriptors

Every process starts with three open fds wired to the terminal:

```
fd 0  stdin    ⟵ keyboard (or < file, or a pipe)
fd 1  stdout   ⟶ terminal (or > file, or | next command)   ← normal output
fd 2  stderr   ⟶ terminal (or 2> file)                      ← errors/diagnostics
```

Keeping stdout and stderr **separate** is deliberate: you can pipe real output onward while still seeing (or separately logging) errors. Confusing the two is a top beginner mistake.

9.2 Redirection operators

Bash

```bash
cmd > file        # stdout → file (TRUNCATE/overwrite)
cmd >> file       # stdout → file (APPEND)
cmd 2> err.log    # stderr → file
cmd > out 2> err  # stdout and stderr to SEPARATE files
cmd > all 2>&1    # BOTH to one file (order matters! see below)
cmd &> all        # bash shorthand for "> all 2>&1"
cmd < input.txt   # feed file as stdin
cmd 2>/dev/null   # discard errors (the "black hole" device)
cmd >/dev/null 2>&1  # discard everything
```

The `2>&1` ordering gotcha — a classic trick question:

Bash

```bash
cmd > file 2>&1      ✓  stdout → file, THEN stderr → "wherever fd1 points" = file. Both in file.
cmd 2>&1 > file      ×  stderr → "wherever fd1 points" = TERMINAL (copied first!),
                        THEN stdout → file. stderr still goes to the terminal!
```

`2>&1` means "make fd 2 a duplicate of *wherever fd 1 currently points*." It captures the target *at that moment*, so the order of redirections matters.

9.3 Pipes

A pipe `|` connects the **stdout of the left** command to the **stdin of the right** command via an in-kernel buffer. Stages run **concurrently**, streaming — not one-then-the-other:

Bash

```bash
ps aux | grep nginx | awk '{print $2}'
#  └ producer ┘   └ filter ┘   └ transform ┘   (all running at once)
```

If a downstream stage exits early (e.g. `… | head`), the upstream gets a **SIGPIPE** on its next write — which is normal and expected.

Bash

```bash
cmd | tee out.log           # SPLIT: write to file AND pass to next stage / screen
cmd | tee -a out.log | …    # append variant
cmd1 | tee >(cmd2) | cmd3   # tee into a process substitution
```

`tee` is the "T-junction": see output live *and* save it.

9.4 Here-docs and here-strings

Feed multi-line or inline text to a command's stdin without a temp file:

```bash
cat <<EOF > config.yml          # HERE-DOC: everything until EOF becomes stdin
server: prod
port: 8080
EOF

cat <<'EOF'                     # quoted 'EOF' → NO variable expansion (literal $HOME)
literal $HOME stays $HOME
EOF

grep foo <<< "$some_variable"   # HERE-STRING: feed one string as stdin
```

9.5 Process substitution `<( )` and `>( )`

Turns the **output of a command into a temporary file-like name** (`/dev/fd/63`), so commands that expect filenames can consume command output directly — no temp files:

```bash
diff <(sort a.txt) <(sort b.txt)        # diff two sorted streams without temp files
comm -13 <(sort old) <(sort new)        # lines only in `new`
while read line; do …; done < <(generate)   # avoid the subshell-loses-variables trap
tar cf - dir/ | tee >(sha256sum)        # compute a checksum while streaming
```

9.6 Named pipes (FIFOs)

An **anonymous pipe** exists only between related processes for the life of the command. A **named pipe (FIFO)** is a persistent filesystem object that *unrelated* processes can open by path — the writer blocks until a reader connects, and vice versa.

```bash
mkfifo /tmp/mypipe
# terminal A:
gzip -c < /tmp/mypipe > out.gz     # reader: blocks waiting for data
# terminal B:
cat big.log > /tmp/mypipe          # writer: streams into the pipe
ls -l /tmp/mypipe                  # shows type 'p'  (prwxr-xr-x)
```

FIFOs are great for decoupling a producer and consumer on one host without a temp file.

Interview takeaway: fd 0/1/2 = stdin/stdout/stderr. `>` overwrites, `>>` appends, `2>` captures errors, and `2>&1` duplicates fd2 onto *wherever fd1 currently points* — so order matters (`> file 2>&1` works, `2>&1 > file` doesn't). Pipes stream concurrently; `tee` splits; `<()` turns command output into a filename; FIFOs let unrelated processes connect.

10 Text Processing (grep/sed/awk)

The Unix text trio plus glue tools. Log triage and ad-hoc data wrangling lean on these constantly.

10.1 grep --- find lines matching a pattern

```bash
grep "ERROR" app.log              # lines containing ERROR
grep -i "error" app.log           # case-insensitive
grep -r "TODO" src/               # recursive through a directory
grep -rn "TODO" src/              # + line numbers
grep -v "DEBUG" app.log           # INVERT: lines NOT matching (drop noise)
grep -c "ERROR" app.log           # COUNT matching lines
grep -o "[0-9]\{3\}" file         # print only the MATCHED part, not whole line
grep -E "404|500|503" access.log  # extended regex (alternation without backslashes)
grep -A3 -B1 "panic" app.log      # 3 lines After, 1 Before each match (context)
grep -w "id" file                 # whole-word match (not "idle", "void")
grep -l "needle" *.log            # just the FILENAMES that contain a match
zgrep "ERROR" app.log.gz          # grep inside gzipped logs without decompressing
```

Regex essentials: `^` start, `$` end, `.` any char, `*` zero-or-more, `[abc]` class, `[^abc]` negated, `\d`-ish via `[0-9]`, `-E` enables `+ ? | ( )` without backslashes.

10.2 sed --- stream editor (substitute, delete, transform)

```bash
sed 's/foo/bar/' file             # replace FIRST foo per line
sed 's/foo/bar/g' file            # replace ALL (global)
sed 's/foo/bar/gi' file           # global + case-insensitive
sed -i 's/foo/bar/g' file         # EDIT IN PLACE (modifies the file!)
sed -i.bak 's/foo/bar/g' file     # in place, keep a .bak backup (safer)
sed -n '10,20p' file              # PRINT only lines 10-20 (-n suppresses default print)
sed '/^#/d' file                  # DELETE comment lines
sed '/^$/d' file                  # delete blank lines
sed 's/[0-9]\+/N/g' file          # replace runs of digits with N
sed 's#/old/path#/new/path#g' f   # use # as delimiter when text has slashes
```

`sed -i` is destructive — always test without `-i` first, or use `-i.bak`.

10.3 awk --- field-aware processing & aggregation

`awk` splits each line into fields (`$1`, `$2`, ... `$NF` = last field; `$0` = whole line) and runs a `pattern { action }` program over every line. It has variables, arrays, arithmetic, and `BEGIN`/`END` blocks — a tiny language.

```bash
awk '{print $1}' access.log              # first field of every line
awk '{print $1, $7}' access.log          # IP and URL (default whitespace split)
awk -F: '{print $1}' /etc/passwd         # field separator = ':' → usernames
awk '$9 == 500' access.log               # lines where 9th field equals 500
awk '$9 >= 400 {print $7}' access.log    # URLs with status ≥ 400
awk 'NR==1{next} {sum+=$3} END{print sum}'    # skip header, sum column 3, print total
awk '{c[$1]++} END{for(k in c) print c[k], k}'  # COUNT occurrences of field 1 (a histogram)
awk '{s+=$NF; n++} END{print s/n}'       # average of the last field
awk 'length($0) > 200' file              # lines longer than 200 chars
```

The `BEGIN { } / { per-line } / END { }` structure is the awk superpower: initialize, accumulate per line, report at the end.

10.4 The glue tools

```bash
cut -d, -f1,3 data.csv          # columns 1 and 3, comma-delimited
sort                            # lexical sort
sort -n                         # NUMERIC sort
sort -rn                        # numeric, reverse (largest first)
sort -k2 -t,                    # sort by 2nd comma-separated key
uniq -c                         # collapse ADJACENT duplicates, prefix with count (needs sort first!)
uniq -d                         # only show duplicated lines
tr 'a-z' 'A-Z'                  # translate/transliterate chars (upcase)
tr -d '\r'                      # delete carriage returns (fix CRLF)
tr -s ' '                       # squeeze repeated spaces into one
wc -l                           # count lines  (-w words, -c bytes)
head -n 20  /  tail -n 20       # first / last 20 lines
tail -f app.log                 # FOLLOW a growing log live
tail -F app.log                 # follow even across log rotation
xargs                           # turn stdin into ARGUMENTS for another command
find . -name '*.log' -mtime +7 # files matching name, modified >7 days ago
jq '.items[].name'              # parse/query JSON (essential for API/k8s output)
```

`uniq` only collapses **adjacent** duplicates — that's why `sort | uniq -c` is the idiom (sort brings duplicates together, `uniq -c` counts them). `xargs` is the bridge between "list of things" and "command that takes args":

```bash
find . -name '*.tmp' | xargs rm          # delete all .tmp files
find . -name '*.tmp' -print0 | xargs -0 rm   # -print0/-0: safe with spaces/newlines in names
echo "1 2 3" | xargs -n1 echo            # one arg per invocation
cat urls.txt | xargs -P8 -n1 curl -sO    # 8 PARALLEL downloads
```

10.5 Real log-analysis one-liners (the money shots)

These are exactly what "analyze this log from the CLI" interview prompts want:

```bash
# Top 10 source IPs hitting an access log
awk '{print $1}' access.log | sort | uniq -c | sort -rn | head

# Count requests by HTTP status code (field 9 in combined log format)
awk '{print $9}' access.log | sort | uniq -n | uniq -c | sort -rn

# Just the 5xx errors, newest, with the URL
awk '$9 ~ /^5/ {print $4, $7, $9}' access.log | tail -20

# Requests per minute (timestamp in field 4 like [17/Jun/2026:14:23:…])
awk '{print substr($4,2,17)}' access.log | uniq -c

# Top 10 URLs by request volume
awk '{print $7}' access.log | sort | uniq -c | sort -rn | head

# Total bytes transferred (field 10), human number
awk '{sum+=$10} END{print sum/1024/1024 " MB"}' access.log

# Error count in the last hour from a syslog-style app log
grep "$(date +'%Y-%m-%dT%H')" app.log | grep -c ERROR

# Approximate p99 of a numeric latency column (field $11), in ms
```

```
awk '{print $11}' access.log | sort -n | awk '{a[NR]=$1} END{print a[int(NR*0.99)]}'

# Unique users who hit a 500 (dedupe IPs among 500s)
awk '$9==500 {print $1}' access.log | sort -u | wc -l

# Slowest 10 requests (sort by latency field, show URL)
sort -k11 -rn access.log | awk '{print $11, $7}' | head
```

Interview takeaway: `grep` filters lines, `sed` edits streams (`-i` = in place, destructive), `awk` does field math and aggregation (`{count[$1]++} END{...}`). The workhorse pipeline `… | sort | uniq -c | sort -rn | head` answers "what are the most common X" — top IPs, top errors, top URLs. Be able to write it without hesitating.

11 Performance Debugging (the USE method)

When a box is unhealthy, **don't guess — measure, methodically.** The framework is Brendan Gregg's **USE method**: for *every resource*, check **U**tilization, **S**aturation, **E**rrors. (§09 covers USE/RED theory; this section is the Linux *tooling* layer.)

```
USE = Utilization + Saturation + Errors,  applied to each resource:

Resource │ Utilization        │ Saturation              │ Errors
─────────┼────────────────────┼─────────────────────────┼──────────────────
CPU      │ mpstat, top  (%)   │ vmstat 'r' (run-queue)  │ dmesg (MCE)
Memory   │ free -h            │ vmstat si/so (swapping) │ dmesg (ECC/OOM)
Disk     │ iostat %util       │ iostat await/aqu-sz     │ smartctl, dmesg
Network  │ sar -n DEV (%)     │ ss -s, dropped/overruns │ netstat -s, ip -s link
```

11.1 Load average --- the most misunderstood number

```
Bash
$ uptime
 14:23:01 up 12 days,  3:42,  2 users,  load average: 4.18, 3.04, 2.91
                                                      └1m┘ └5m┘ └15m┘
```

Load average is the **number of processes in state R (running/runnable) OR D (uninterruptible sleep)**, averaged over 1/5/15 minutes. Two crucial consequences:

1. **Compare to core count.** On an 8-core box, load 4 = ~50% busy (fine); on a 2-core box, load 4 means tasks are queuing (twice the work the CPU can do at once). `load > cores` is *not always* a problem and `load < cores` is *not always* healthy.
2. **Load counts D-state (I/O wait), not just CPU.** A box can show **load 20 with the CPU 95% idle** because 20 processes are all stuck in `D` waiting on a dying disk. High load ≠ high CPU. This is the single most important load-average insight.

```
Bash
nproc                # number of cores (the yardstick for load)
cat /proc/loadavg    # raw: 4.18 3.04 2.91 5/812 99123  (running/total, last PID)
```

11.2 The tools, and what each *reveals*

```bash
# ── CPU ──
top                 # live per-process CPU/mem; press 1 = per-core, look at %us %sy %id %wa %st
htop                # friendlier top (tree, scroll, F9 kill)
mpstat -P ALL 1     # per-CORE utilization → spot a single saturated core (single-threaded bottleneck)
vmstat 1            # 'r'=run-queue (CPU saturation), 'us/sy/id/wa', 'si/so' (swap) — one-stop
pidstat 1           # per-process CPU/IO/mem over time

# top's CPU line decoded:
#   %us user   %sy kernel/system   %id idle   %wa I/O-WAIT (key!)   %st stolen (noisy neighbor/VM)

# ── Memory ──
free -h             # used / free / available / swap. 'available' is the real headroom number.
vmstat 1            # si/so columns: nonzero = ACTIVELY SWAPPING = memory pressure → slowness
cat /proc/meminfo   # detailed breakdown (Buffers, Cached, SwapTotal…)
slabtop             # kernel slab cache usage (kernel memory leaks)

# ── Disk I/O ──
iostat -xz 1        # per-device: %util (busy), await (latency ms), aqu-sz (queue depth), r/s w/s
iotop               # which PROCESS is doing the I/O
df -h ; df -i       # space ; inodes

# ── Network ──
ss -tunap           # sockets: TCP/UDP, numeric, all states, owning process (replaces netstat)
ss -s               # socket summary counts by state
netstat -s          # protocol stats: retransmits, drops, overruns (errors!)
sar -n DEV 1        # per-interface throughput & %util
nstat               # delta of kernel net counters

# ── Open files / what a process is touching ──
lsof -p <pid>       # every file/socket the process has open
lsof -i :8080       # who is listening on / connected to port 8080
lsof +D /var/log    # who has files open under a directory
lsof | grep deleted  # deleted-but-open files (disk-full mystery)

# ── Syscall / library tracing (why is it hung?) ──
strace -p <pid>     # live syscalls of a running process (attach)
strace -f -T -tt cmd # follow forks, time each call, timestamps
strace -c cmd       # SUMMARY: which syscalls dominate, error counts
ltrace -p <pid>     # library calls instead of syscalls

# ── CPU profiling / flame graphs ──
perf top            # live function-level CPU hotspots
perf record -g -p <pid> -- sleep 30 ; perf report   # sampled stacks → flame graph input

# ── Kernel ring buffer & the OOM killer ──
dmesg -T | tail     # hardware errors, OOM kills, segfaults (-T = human timestamps)
dmesg -T | grep -i -E 'oom|killed process|out of memory'
journalctl -k       # kernel log via systemd
```

11.3 The OOM killer

When the kernel runs out of memory (and swap) and a process demands more, it can't say ”no” gracefully to an already-running program — so the **Out-Of-Memory killer** picks a victim and SIGKILLs it to reclaim memory. It scores processes by an `oom_score` (roughly: memory footprint, adjusted by `oom_score_adj`). The victim dies instantly (SIGKILL → no cleanup), which often looks like a mysterious crash. The fingerprint is in `dmesg`:

```
$ dmesg -T | grep -i oom
[Wed Jun 17 14:05:33 2026] Out of memory: Killed process 4823 (java) total-vm:8.2GB...
```

```bash
cat /proc/<pid>/oom_score          # current OOM score
echo -1000 > /proc/<pid>/oom_score_adj   # protect a critical process from the OOM killer
```

In containers, an OOM kill driven by a **cgroup memory limit** is what surfaces as Kubernetes `OOMKilled` — the pod hit its memory ceiling, not the host's.

Interview takeaway: Drive triage with USE: per resource, check Utilization, Saturation, Errors. Load average counts R *and* D, so high load with idle CPU means I/O wait, not CPU. `vmstat`/`mpstat` for CPU, `free`/`vmstat si/so` for memory pressure, `iostat -x` (await/%util) for disk, `ss`/`sar` for network, `lsof`/`strace` to see what a process is actually doing, and `dmesg` for OOM kills and hardware errors.

12 Systemd, Cron & Services

12.1 systemd --- the modern init & service manager

`systemd` is PID 1 on most distros. It boots the system, manages **units** (services, sockets, timers, mounts), tracks dependencies, restarts crashed services, and centralizes logging via the **journal**.

A service is described by a **unit file** (`/etc/systemd/system/myapp.service`):

```ini
[Unit]
Description=My API service
After=network.target               # start ordering: after network is up
Requires=postgresql.service        # hard dependency

[Service]
ExecStart=/usr/bin/myapp --port 8080
ExecReload=/bin/kill -HUP $MAINPID # how a `reload` is performed
Restart=on-failure                 # auto-restart if it crashes
RestartSec=5
User=myapp                         # run as an UNPRIVILEGED user (least privilege)
MemoryMax=512M                     # cgroup memory cap (OOM-kills only this service)
CPUQuota=50%                       # cgroup CPU cap
TimeoutStopSec=30                  # SIGTERM grace before SIGKILL (graceful shutdown!)

[Install]
WantedBy=multi-user.target         # enable target → starts at boot
```

```bash
systemctl start   myapp            # start now
systemctl stop    myapp            # SIGTERM, then SIGKILL after TimeoutStopSec
systemctl restart myapp
systemctl reload  myapp            # ExecReload (e.g. SIGHUP) — no full restart
systemctl enable  myapp            # start at boot
systemctl status  myapp            # state, PID, recent log lines, memory/cpu
systemctl daemon-reload            # re-read unit files after editing them
systemctl list-units --failed      # what's broken
```

systemd runs each service in its **own cgroup**, so `MemoryMax`/`CPUQuota` are enforced by the kernel, and the service shuts down gracefully (SIGTERM → grace → SIGKILL) for free.

12.2 journalctl --- the systemd log

```bash
journalctl -u myapp                # all logs for one unit
journalctl -u myapp -f             # FOLLOW live (like tail -f)
journalctl -u myapp --since "10 min ago"
journalctl -u myapp -p err         # only error-priority and above
journalctl -u myapp -b             # since last boot
journalctl -k                      # kernel messages (dmesg equivalent)
journalctl --disk-usage            # how big the journal is
journalctl --vacuum-time=7d        # trim journal older than 7 days (free disk!)
```

12.3 Cron and systemd timers

Two ways to run scheduled jobs.

cron — classic, file-based:

```bash
crontab -e        # edit YOUR user's cron table
crontab -l        # list

# ┌ min   ┌ hour  ┌ day-of-month  ┌ month  ┌ day-of-week    command
# *       *       *               *        *
  0       2       *               *        *     /usr/local/bin/backup.sh   # daily 02:00
  */15    *       *               *        *     /usr/local/bin/healthcheck # every 15 min
  0       0       *               *        0     /usr/local/bin/weekly.sh   # Sun midnight
  @reboot                                        /usr/local/bin/warmup.sh   # at boot
```

Gotchas that bite people: cron runs with a **minimal environment** (sparse `PATH`, no shell profile), so use **absolute paths** and set env explicitly. Cron emails stdout/stderr to the user unless you redirect (`>> /var/log/job.log 2>&1`). System-wide jobs live in `/etc/cron.d/`, `/etc/cron.daily/`, etc.

systemd timers — the modern alternative (better logging via journal, dependency-aware, supports `Persistent=` to catch up missed runs after downtime):

```ini
# backup.timer
[Timer]
OnCalendar=*-*-* 02:00:00
Persistent=true            # run on next boot if the machine was off at 02:00
[Install]
WantedBy=timers.target
```

12.4 Resource limits: ulimits and sysctl

ulimits are per-process resource ceilings. The famous one is **open files**:

```bash
ulimit -a              # all current limits
ulimit -n              # max open file descriptors (default often 1024 — too low for servers!)
ulimit -n 65536        # raise (soft limit, up to the hard limit)
```

```
ulimit -u               # max user processes (fork-bomb protection)
ulimit -c unlimited     # enable core dumps for debugging crashes
```

”**Too many open files**” (`EMFILE`) is a very common production error: a high-concurrency server (or one leaking fds) hits the 1024 default. Fix in the systemd unit (`LimitNOFILE=65536`) or `/etc/security/limits.conf`.

sysctl tunes kernel parameters (the `/proc/sys` tree):

```bash
sysctl vm.swappiness                    # how aggressively to swap (0-100)
sysctl -w vm.swappiness=10              # lower → keep app pages in RAM, swap less
sysctl net.core.somaxconn               # max queued connections (raise for busy servers)
sysctl net.ipv4.tcp_tw_reuse
sysctl -p                               # apply /etc/sysctl.conf persistently
```

Swappiness (0–100) controls the kernel's eagerness to swap anonymous memory to disk to grow the page cache. Latency-sensitive services (databases, JVM apps) set it **low** (`1`–`10`) so their working set stays in RAM instead of being paged to slow disk — a paged-out heap causes brutal GC pauses (see §09).

Interview takeaway: systemd manages services as units in their own cgroups — `systemctl` to control, `journalctl -u <svc> -f` to tail logs, `Restart=on-failure` + `TimeoutStopSec` for resilient graceful shutdown, `MemoryMax`/`CPUQuota` for limits. Schedule with cron (mind the bare environment — use absolute paths) or systemd timers. Raise `ulimit -n` to dodge ”too many open files”; lower `vm.swappiness` to keep hot pages off disk.

13 Containers Under the Hood (namespaces & cgroups)

A container is **not a VM**. There is no guest kernel — every container on a host shares the *one* host kernel. ”Container” is just a regular Linux process (or process tree) wrapped in two kernel features:

```
┌──────────────────────────────────────────────┐
ISOLATION = what a process can SEE     →  NAMESPACES
LIMITS    = how much it can USE        →  CGROUPS
└──────────────────────────────────────────────┘
Docker / containerd / Kubernetes are orchestration on top of these.
```

13.1 Namespaces --- isolating what a process can see

A **namespace** virtualizes a global kernel resource so the processes inside it see their own private instance. The kinds:

Namespac	Isolates	Effect inside the container
PID	Process IDs	Container's main process is **PID 1**; can't see host processes.
NET	Network stack	Own interfaces, IPs, routing table, ports (own `eth0`, own port 80).
MNT	Mount points	Own filesystem view / root (`chroot` on steroids).
UTS	Hostname & domain	Own `hostname` independent of the host.
IPC	SysV IPC, shared mem, message queues	Can't see host's IPC objects.

Namespac	Isolates	Effect inside the container
USER	UID/GID mappings	**root inside (UID 0)** can map to an **unprivileged UID outside** — key security win.
CGROUP	cgroup root view	Container sees its own cgroup hierarchy.

You can create them by hand — proving Docker isn't magic:

```bash
unshare --pid --fork --mount-proc bash   # new PID namespace
ps aux        # inside: this bash is PID 1 — it can't see host processes!

unshare --uts bash
hostname container-x   # changes hostname only inside; host is unaffected

lsns                   # list all namespaces on the host
ls -l /proc/<pid>/ns   # a process's namespace memberships (inode per ns)
```

13.2 cgroups --- limiting how much a process can use

Control groups (cgroups) account for and *cap* resource usage for a group of processes — CPU, memory, I/O, PIDs. This is how a container is told "you get 0.5 CPU and 512 MB."

```
cgroup v2 (unified hierarchy under /sys/fs/cgroup):
  memory.max          → hard memory cap. Exceed it → cgroup OOM kill (K8s "OOMKilled")
  memory.high         → soft cap (throttle reclaim before the hard kill)
  cpu.max             → "50000 100000" = 50ms CPU per 100ms = 0.5 cores  (THROTTLING)
  io.max              → disk I/O bandwidth/IOPS caps
  pids.max            → cap number of processes (fork-bomb protection)
```

```bash
cat /sys/fs/cgroup/<...>/memory.max        # the memory ceiling
cat /sys/fs/cgroup/<...>/cpu.stat          # nr_throttled, throttled_time (CPU throttling!)
systemd-cgtop                              # live per-cgroup resource usage
```

Two container behaviors interviewers love, both explained by cgroups:

- **CPU throttling:** hit `cpu.max` and the kernel *pauses* your process until the next period. Your app isn't crashing — it's being **throttled**, showing as latency spikes with the CPU not pegged. Diagnose via `nr_throttled`/`throttled_time` in `cpu.stat`.
- **`OOMKilled`:** exceed `memory.max` and the **cgroup OOM killer** SIGKILLs the offending process *inside that cgroup* (the host has plenty of RAM). The pod restarts; `dmesg` shows the kill.

13.3 Putting it together: what `docker run` actually does

```
docker run -m 512m --cpus=0.5 nginx
       │
       ├─ create NEW namespaces (PID, NET, MNT, UTS, IPC, [USER])
       │     → isolated process, own network, own root fs (the image layers)
       ├─ create a CGROUP with memory.max=512M, cpu.max=0.5 core
       ├─ pivot_root into the image's filesystem (overlayfs union of layers)
       ├─ drop capabilities, apply seccomp profile (restrict syscalls)
       └─ exec the entrypoint as PID 1 inside the namespaces
```

The image is a stack of read-only layers unioned by **overlayfs** with a writable top layer — that's why containers start instantly (no copy) and why writes vanish on removal unless you

mount a volume.

> **PID 1 pitfall:** the entrypoint becomes PID 1, which by default does **not** reap orphaned children and has non-standard signal behavior. A long-running container whose PID 1 doesn't reap will accumulate **zombies**; one that doesn't forward SIGTERM won't shut down gracefully. The fix is a tiny init like `tini` (`docker run --init`) as PID 1.

Interview takeaway: A container = a normal Linux process isolated by *namespaces* (what it can SEE: PID/NET/MNT/UTS/IPC/USER) and capped by *cgroups* (what it can USE: cpu.max/memory.max). No guest kernel — the host kernel is shared. Kubernetes `OOMKilled` = hit cgroup `memory.max`; mysterious latency with idle CPU in a container = `cpu.max` *throttling*. Run a proper init as PID 1 to reap zombies and forward SIGTERM.

14 Shell Scripting Essentials

Bash is the glue of ops. Reliable scripts come from a handful of disciplines.

14.1 Variables, quoting, and the #1 bug

```bash
name="Alice"            # NO spaces around = (assignment)
echo "$name"            # use → $name or ${name}
echo "${name}_suffix"   # braces disambiguate
count=$(ls | wc -l)     # command substitution → capture output
files=(*.log)           # array
echo "${files[@]}"      # all elements
```

Always quote your variables. Unquoted `$var` undergoes word-splitting and glob expansion — the cause of countless catastrophic bugs:

```bash
file="my report.txt"
rm $file                # ✗ runs: rm my  report.txt   → deletes TWO wrong files
rm "$file"              # ✓ runs: rm "my report.txt"  → correct
```

14.2 Conditionals

```bash
if [[ -f "$path" ]]; then echo "file exists"; fi
if [[ -d "$dir" ]];  then echo "directory"; fi
if [[ -z "$var" ]];  then echo "empty/unset"; fi
if [[ -n "$var" ]];  then echo "non-empty"; fi
if [[ "$a" == "$b" ]]; then …; fi          # string equality
if [[ "$n" -gt 10 ]];  then …; fi          # numeric: -eq -ne -lt -le -gt -ge
if [[ "$s" =~ ^[0-9]+$ ]]; then …; fi      # regex match
if grep -q ERROR log; then …; fi           # branch on a command's EXIT CODE
```

Prefer `[[ … ]]` (bash conditional) over `[ … ]` (the older `test`) — it's safer with empty vars and supports `=~`, `&&`, `||`.

14.3 Loops

```bash
for f in *.log; do echo "$f"; done
for i in {1..5}; do echo "$i"; done
for ((i=0; i<5; i++)); do echo "$i"; done

while read -r line; do            # read a file/stream line by line (-r: don't mangle backslashes)
  echo "got: $line"
done < input.txt

while IFS=, read -r col1 col2; do  # parse CSV
  echo "$col1 → $col2"
done < data.csv
```

14.4 Functions and exit codes

```bash
log() { echo "[$(date +%T)] $*" >&2; }     # helper that logs to stderr

deploy() {
  local target="$1"                        # 'local' keeps scope clean
  build || return 1                        # propagate failure
  push "$target"
}

deploy prod
echo "exit code: $?"        # $? = exit status of last command (0 = success)
```

`$?` is 0 on success, non-zero on failure. Chain on it:

```bash
make && ./run                # run ONLY if make succeeded
backup || alert "backup failed"   # alert ONLY if backup failed
cmd1; cmd2                   # run sequentially regardless
```

14.5 The safety preamble: `set -euo pipefail`

Put this at the top of every serious script:

```bash
#!/usr/bin/env bash
set -euo pipefail
IFS=$'\n\t'
```

Flag	Effect	Why
`set -e`	Exit immediately on any command failure	Don't barrel past errors corrupting state
`set -u`	Error on use of an **unset** variable	Catches typos like `rm -rf "$PREFXI/"` (→ `rm -rf /`)
`set -o pipefail`	A pipeline fails if **any** stage fails	Without it, 'false

Flag	Effect	Why
`set -x`	Print each command before running (debug)	Trace what actually executed

Without `set -u`, a typo'd variable expands to empty and `rm -rf "$DIR/"` becomes `rm -rf /` — a legendary disaster. `set -e` + `pipefail` make scripts fail loud and early instead of silently doing the wrong thing.

14.6 A small, real script (with traps and safety)

Bash

```bash
#!/usr/bin/env bash
# rotate_and_backup.sh — compress yesterday's logs, upload, clean up. Safely.
set -euo pipefail

LOG_DIR="/var/log/myapp"
DEST="s3://backups/myapp"
WORK="$(mktemp -d)"                      # private temp dir
trap 'rm -rf "$WORK"' EXIT               # guaranteed cleanup on ANY exit
trap 'echo "interrupted, aborting" >&2; exit 130' INT TERM

log() { echo "[$(date -Is)] $*" >&2; }

yesterday="$(date -d 'yesterday' +%Y-%m-%d)"
shopt -s nullglob                        # empty glob → no match, not literal '*'
files=("$LOG_DIR"/*"$yesterday"*.log)

if (( ${#files[@]} == 0 )); then
  log "no logs for $yesterday, nothing to do"
  exit 0
fi

log "compressing ${#files[@]} files"
for f in "${files[@]}"; do
  gzip -c "$f" > "$WORK/$(basename "$f").gz"
done

log "uploading to $DEST"
if aws s3 cp "$WORK" "$DEST/$yesterday/" --recursive; then
  log "upload ok; removing local originals"
  rm -f "${files[@]}"
else
  log "upload FAILED; keeping local logs" >&2
  exit 1
fi
log "done"
```

This shows every essential: `set -euo pipefail`, `trap … EXIT` for cleanup, `trap … INT TERM` for graceful interruption, quoted variables, arrays, `mktemp`, exit codes, logging to stderr.

Interview takeaway: Quote every variable (`"$var"`) to avoid word-splitting disasters; start scripts with `set -euo pipefail` so they fail fast instead of silently corrupting state; use `trap '…' EXIT` for guaranteed cleanup and `trap … TERM INT` for graceful shutdown; `$?` carries the exit code (0 = success) for `&&`/`||` chaining.

15 Networking from the CLI

The triage toolkit for "is it the network?" (TCP/IP theory lives in §03).

15.1 Interfaces, addresses, routes --- `ip`

The modern replacement for `ifconfig`/`route`:

```bash
ip a                    # all interfaces and their IP addresses (ip addr)
ip link                 # link-layer state (up/down, MAC, MTU)
ip route                # the routing table — where does traffic go?
ip route get 8.8.8.8    # which route/interface a destination would use
ip -s link              # per-interface stats: errors, drops, overruns (the 'E' in USE)
ip neigh                # ARP table (neighbor cache)
```

15.2 Sockets & ports --- `ss`

`ss` (replaces `netstat`) shows who's listening and who's connected:

```bash
ss -tlnp                # TCP, Listening, Numeric, with Process → "what's listening on what port"
ss -tunap               # TCP+UDP, all states, numeric, process
ss -s                   # summary counts by state
ss -t state established # only established TCP connections
ss dst :443             # connections to port 443
```

This answers "**address already in use**": `ss -tlnp | grep :8080` shows the PID already bound to 8080. (Often a TIME_WAIT socket from a prior crash, or a stale instance still running.)

15.3 DNS --- `dig`

```bash
dig example.com                 # full DNS answer
dig +short example.com          # just the IP(s)
dig example.com MX              # mail records
dig @8.8.8.8 example.com        # query a SPECIFIC resolver (bypass local)
dig +trace example.com          # follow delegation from the root (debug resolution)
nslookup example.com            # simpler/legacy
getent hosts example.com        # resolution the way the SYSTEM sees it (respects /etc/hosts, nsswitch)
```

`dig +short` vs `getent` matters: `dig` talks DNS directly; `getent`/the app go through `/etc/hosts` + `nsswitch.conf` first — so they can disagree, which explains "works with dig but the app can't resolve it."

15.4 HTTP & timing --- `curl`

```bash
curl -I https://example.com              # headers only (HEAD)
curl -v https://example.com              # verbose: TLS handshake, request/response
curl -s -o /dev/null -w "%{http_code}\n" URL   # just the status code
curl -L URL                              # follow redirects
```

```
curl -X POST -d '{"k":"v"}' -H 'Content-Type: application/json' URL

# Latency breakdown — WHERE the time goes (DNS vs connect vs TLS vs server):
curl -s -o /dev/null -w \
 'dns:%{time_namelookup} connect:%{time_connect} tls:%{time_appconnect} ttfb:%{time_starttransfer}
 ↪  total:%{time_total}\n' \
 https://example.com
```

That `-w` timing string is a senior move: it tells you whether slowness is **DNS**, **TCP connect**, **TLS handshake**, or **server processing (TTFB)** — without a packet capture.

15.5 Connectivity & path --- `ping`, `traceroute`, `nc`, `mtr`

Bash

```
ping -c4 8.8.8.8            # reachability + RTT (ICMP)
traceroute example.com      # the hop-by-hop path; where latency/loss appears
mtr example.com             # ping + traceroute combined, live — best for intermittent loss
nc -zv host 5432            # is the TCP PORT open? (-z scan, -v verbose) — no app protocol needed
nc -l 9000                  # listen on a port (quick test server)
echo "ping" | nc host 9000 # send raw bytes to a service
telnet host 80              # crude TCP probe (then type an HTTP request)
```

`nc -zv host port` is the fastest "can I even reach the port?" check — distinguishing a network/firewall problem (port unreachable) from an application problem (port open but app broken).

15.6 Packet capture --- `tcpdump`

The ground truth when nothing else explains it:

Bash

```
tcpdump -i any -n port 443                # all traffic on 443, numeric (no DNS)
tcpdump -i eth0 host 10.0.0.5 -c 100      # 100 packets to/from a host
tcpdump -i any 'tcp[tcpflags] & tcp-syn != 0'  # SYNs only (connection attempts)
tcpdump -i any -w capture.pcap            # write to file → open in Wireshark
```

Use it to confirm packets actually arrive (firewall? routing?), to see TCP resets, retransmits, or to prove the request never left the box.

Interview takeaway: `ip a`/`ip route` for interfaces & routing, `ss -tlnp` for "what's listening / port already in use", `dig +short`/`getent` for DNS (and they can disagree — DNS vs `/etc/hosts`), `curl -w` to localize latency (DNS vs connect vs TLS vs TTFB), `nc -zv host port` to test a port without the app, and `tcpdump` for ground-truth packets.

16 Worked Debugging Scenarios

These are the on-call narratives interviews simulate. Each shows the *reasoning* and the *exact* commands, in order.

16.1 Scenario 1 --- "The server is slow." (systematic sweep)

Narrate a structured USE sweep; never start randomly.

```bash
# 0. Frame it: what's slow, since when, what changed?
uptime                          # load avg vs nproc — is it CPU or queueing?
nproc

# 1. CPU
top                             # %us vs %sy vs %wa vs %st ; which process is hot?
mpstat -P ALL 1                 # is ONE core pegged (single-threaded bottleneck)?
#   → high %us  → app CPU-bound (profile it: perf top / flame graph)
#   → high %sy  → syscall/kernel heavy (strace -c the hot pid)
#   → high %wa  → NOT cpu — jump to disk (step 3)
#   → high %st  → noisy VM neighbor / throttling (cloud)

# 2. Memory
free -h                         # is 'available' tiny? is swap used?
vmstat 1                        # si/so nonzero → ACTIVELY swapping → slowness
dmesg -T | grep -i oom          # did the OOM killer fire?

# 3. Disk
iostat -xz 1                    # %util ~100%? await high (ms)? aqu-sz growing?
iotop                           # which process is hammering the disk?

# 4. Network
ss -s                           # socket counts; TIME_WAIT/SYN-RECV pileups?
sar -n DEV 1                    # NIC saturated? errors/drops?
netstat -s | grep -i retrans    # TCP retransmits → packet loss

# 5. The hot process
ps aux --sort=-%cpu | head
strace -c -p <pid>              # what syscalls dominate? errors?

# 6. Logs
journalctl -u myapp --since "15 min ago" -p warning
tail -n 200 /var/log/myapp/error.log
dmesg -T | tail
```

The discipline (CPU → mem → disk → net → process → logs) is what's being graded, more than any single command.

16.2 Scenario 2 --- "Disk is at 100%." (and the deleted-open-file trap)

```bash
df -h                           # confirm WHICH filesystem is full (/ ? /var ?)
df -i                           # is it INODES, not bytes? (millions of tiny files)

# Find the space hogs, staying on one filesystem:
du -xh --max-depth=1 / | sort -rh | head
du -xh /var/log | sort -rh | head        # logs are the usual culprit

# If df says 100% but du finds nothing big → DELETED-BUT-OPEN file:
lsof | grep -i deleted                   # process still holding a removed file
#   COMMAND  PID  USER  FD   ...  SIZE   ...  NAME
#   java     812  app   7w   ...  9.0G   ...  /var/log/app.log (deleted)

# Fix WITHOUT another rm (the name is gone). Options:
: > /proc/812/fd/7              # truncate the still-open fd → space freed instantly
# or signal the process to reopen its log:
kill -HUP 812                   # many daemons reopen logs on SIGHUP (logrotate uses this)
# or restart the service.
```

Bonus root cause: a missing/broken **logrotate** config let a log grow unbounded, or an app logs at DEBUG in prod.

16.3 Scenario 3 --- "High load, but the CPU is idle." (I/O wait / D-state)

```bash
uptime                          # load average 25.0 ...
mpstat 1                        # but %idle is 90%! CPU isn't the problem.
                                # look at %iowait — high → blocked on disk

# Load counts R *and* D. Find the D-state (uninterruptible) processes:
ps -eo pid,state,wchan:25,comm | awk '$2=="D"'
#   PID  S  WCHAN              COMMAND
#   913  D  io_schedule        postgres   ← stuck waiting on disk I/O

iostat -xz 1                    # %util pinned at 100, await = hundreds of ms → dying/slow disk
dmesg -T | grep -i -E 'i/o error|ata|nfs|timeout'   # disk/NFS errors?
```

Diagnosis: the disk (or an NFS mount) is the bottleneck; processes pile up in `D`, inflating load while the CPU sits idle. `kill -9` won't free a D-state process — you must fix the I/O (failing disk, stalled storage, dead NFS server).

16.4 Scenario 4 --- "Memory leak / OOM kill."

```bash
free -h                         # available shrinking over time
dmesg -T | grep -i -E 'oom|killed process'   # confirm an OOM kill happened
#   Out of memory: Killed process 4823 (java) total-vm:8.2GB, anon-rss:7.9GB...

# Watch the suspect grow:
ps -o pid,rss,vsz,comm -p <pid>     # RSS climbing monotonically = leak signature
watch -n5 'ps -o rss= -p <pid>'

# Per-process detail:
cat /proc/<pid>/status | grep -E 'VmRSS|VmSwap|Threads'
pmap -x <pid> | tail            # memory map breakdown

# In a container: is it a cgroup limit, not host RAM?
cat /sys/fs/cgroup/<...>/memory.max
cat /sys/fs/cgroup/<...>/memory.events   # 'oom_kill' counter
```

For a JVM/managed runtime, "leak" means retained references; take a heap dump and analyze (see §09). For native code, RSS climbing without bound points to unfreed allocations. Mitigation while you fix: raise the limit, lower `oom_score_adj` for critical processes, or set a `MemoryMax` so only the offender dies.

16.5 Scenario 5 --- "A process is hung." (strace / lsof)

```bash
ps -o pid,state,wchan,comm -p <pid>    # state R? S? D? what's it waiting on (wchan)?

strace -p <pid>                 # attach: what syscall is it stuck in RIGHT NOW?
#   read(8,  <stuck here>       → blocked reading fd 8 (a slow/dead peer?)
#   futex(0x..., FUTEX_WAIT…)   → waiting on a lock (deadlock/contention)
#   <no output at all>          → not making syscalls (busy-looping in userspace → perf top)

```

```
lsof -p <pid>                  # what is fd 8? a socket? a file? a pipe?
#   → if it's a TCP socket: who's the peer? is it ESTABLISHED but silent?
ls -l /proc/<pid>/fd/8         # resolve the fd to its target

# If it's a deadlock (futex), get all thread stacks:
cat /proc/<pid>/task/*/stack   # kernel stacks per thread
# (for the JVM: jstack <pid>; for Go: SIGQUIT dumps goroutine stacks)
```

`strace` showing a blocking `read`/`recvfrom`/`futex` tells you *exactly* what the process is waiting for; `lsof`/`/proc/<pid>/fd` tells you *what* that fd is. A silent `strace` (no syscalls) means it's spinning in userspace — switch to `perf top -p <pid>`.

16.6 Scenario 6 --- "Port already in use" / connection issues.

Bash

```
# "bind: address already in use" when starting on :8080
ss -tlnp | grep :8080          # WHO is bound? → PID + program
#   LISTEN 0 128 *:8080 *:* users:(("old_app",pid=4501,fd=6))
kill 4501                      # stop the stale holder (graceful first)
# or it's a lingering TIME_WAIT: enable SO_REUSEADDR in the app, or wait it out.

# "connection refused" reaching another service
nc -zv db.internal 5432        # is the PORT even open? refused = nothing listening / firewall
ss -tlnp | grep 5432           # (on the server) is the service actually listening on that interface?
#   bound to 127.0.0.1:5432 instead of 0.0.0.0 → only local connects work!
dig +short db.internal         # does the name resolve? to the right IP?
ip route get <ip>              # is there a route to it?
curl -v http://api.internal/health   # if HTTP: see TLS/headers/timeouts
tcpdump -i any -n host <ip> and port 5432   # do packets actually flow? SYN with no SYN-ACK =
↪  firewall/down
```

The decision tree: **resolve** (dig) → **route** (ip route get) → **reach the port** (nc) → **app responds** (curl) → **packets flow** (tcpdump). A common gotcha: a service bound to `127.0.0.1` instead of `0.0.0.0` accepts only local connections, so remote clients get "connection refused" while local curl works.

Interview takeaway: Every scenario rewards a *narrated, structured* approach: frame the symptom, sweep resources in order (CPU→mem→disk→net), and reach for the right probe (`lsof` for deleted-open files and fds, `D`-state + `iostat` for I/O-bound load, `dmesg` for OOM, `ss` for port conflicts, `strace` for hangs). Knowing the deleted-but-open-file and bound-to-127.0.0.1 gotchas signals real operational experience.

17 Architecture / Diagrams

17.1 The user/kernel boundary and a syscall

```
USERLAND                                KERNEL

  your process
      │  read(fd, buf, n)        glibc wrapper
      │ ─────────────────────►   sets up registers, executes `syscall`
      │                                 │  trap → ring 0
      │                                 ▼
      │                          sys_read():
```

```
      │                          check fd, permissions
      │                          copy data from page cache (or block on disk → state D)
      │                          return byte count
      │  ◄──────────────────────────  return to ring 3
      ▼
  continue with data in buf
```

17.2 A pipeline's data flow

```
[ access.log ]
     │ cat
     ▼ stdout
   ──┤ grep ERROR ├── stdout
                 ▼ stdin
               ──┤ awk '{print $7}' ├── stdout
                                   ▼ stdin
                                 ──┤ sort ├──┤ uniq -c ├──┤ sort -rn ├──┤ head ├──► terminal

Each '|' = an in-kernel buffer wiring left.stdout → right.stdin.
All stages run CONCURRENTLY, streaming. Downstream exit → upstream gets SIGPIPE.
```

17.3 File descriptors & redirection

```
                ┌──────────── process ────────────┐
keyboard ──►    │ fd0 stdin                       │
                │ fd1 stdout ──► terminal         ├──►  > file   (overwrite)
                │ fd2 stderr ──► terminal         ├──►  >> file  (append)
                └─────────────────────────────────┘──►  2> err   (errors aside)
                                                  └►  2>&1     (fd2 := wherever fd1 points NOW)
< file feeds fd0.   | wires fd1 → next process's fd0.
```

17.4 Inode, directory entry, links

```
DIRECTORY /home/u (itself an inode, type d)
  ┌──────────────────────────┐
  │ "report.txt"  → inode 42 ├─┐
  │ "backup.txt"  → inode 42 ├─┤   (hard link: 2 names, 1 inode, link_count=2)
  │ "shortcut"    → inode 77 │ │
  └──────────────────────────┘ │
                               ▼
                        INODE 42  (link_count=2)
                         mode/owner/size/times
                         block ptrs → [data blocks on disk]

  inode 77 (a symlink): its DATA is the text "/home/u/report.txt"
       → resolved by NAME at access time; dangles if target removed.

rm "backup.txt"  → unlink → link_count 2→1, data UNTOUCHED.
rm "report.txt"  → link_count 1→0; if also no open fd → blocks freed.
```

17.5 Container = namespaces + cgroups

```
            HOST KERNEL (single, shared)
  ┌──────────────────────────────────────────────────┐
  │  Container A            Container B              │
  │  ┌──────────────┐       ┌──────────────┐         │
  │  │ ns: pid/net/ │       │ ns: pid/net/ │  ← SEE (isolation)
  │  │     mnt/uts… │       │     mnt/uts… │         │
  │  │ PID 1 = app  │       │ PID 1 = app  │         │
  │  └──────────────┘       └──────────────┘         │
  │  cgroup: mem.max=512M   cgroup: cpu.max=0.5  ← USE (limits)
```

```
└──────────────────────────────────────────────┘
No guest kernels. Just isolated, capped processes.
```

17.6 Triage decision tree

```
                "Server is slow"
                       │
                       ▼
        ┌──────────────┴──────────────┐
load > nproc?                  load ≈ low?
      │                              │
is CPU busy? ──── no ──► high %wa? ──► DISK (iostat, iotop, D-state)
      │ yes                          │
%us high → app CPU-bound             └─ swap si/so? ──► MEMORY (free, OOM in dmesg)
%sy high → syscall-heavy
%st high → VM steal/throttle
      │
none of the above + many sockets ──► NETWORK (ss, sar, retrans)
```

18 Real-World Examples

- **logrotate + SIGHUP.** Production logging relies on `logrotate`: it renames `app.log` → `app.log.1`, then sends the daemon `SIGHUP` (or `USR1`) so it **reopens** its log file. Without that reopen, the daemon keeps writing to the now-renamed inode (the deleted-open-file problem in slow motion) and the new `app.log` stays empty.
- **Graceful deploys (Kubernetes).** A rolling update sends each pod **SIGTERM**, waits `terminationGracePeriodSeconds`, then **SIGKILL**. A service that traps SIGTERM, fails its readiness probe (so the load balancer drains it), finishes in-flight requests, and exits → zero dropped connections. One that ignores SIGTERM → connections reset mid-request on every deploy.
- **`OOMKilled` pods.** A Java service with `-Xmx` set higher than the pod's cgroup `memory.max` gets the **cgroup OOM killer** — `kubectl describe pod` shows `Reason: OOMKilled`, `dmesg` shows the SIGKILL. The fix is aligning JVM heap + overhead to the limit, not "give it more memory" reflexively.
- **CPU throttling latency.** A latency-sensitive container with `--cpus=0.5` shows periodic p99 spikes while host CPU sits idle. `cpu.stat` reveals `nr_throttled` climbing — the kernel is *pausing* it each period. Raising the CPU limit (or removing it for latency-critical workloads) fixes the spikes.
- **"Too many open files."** A high-throughput proxy hits the default `ulimit -n 1024` under load → `accept: too many open files`, connections rejected. Raising `LimitNOFILE=1048576` in the systemd unit resolves it. A *leaking* fd count (not closing sockets) presents identically — `ls /proc/<pid>/fd | wc -l` climbing distinguishes the two.
- **Log triage during an incident.** `awk '$9 ~ /^5/ {print $7}' access.log | sort | uniq -c | sort -rn | head` instantly shows *which endpoints* are throwing 5xx — turning "the site is erroring" into "the `/checkout` endpoint is failing 3000×/min."
- **Cron's bare environment.** A backup script that works by hand but silently fails under cron — because cron's `PATH` lacks `/usr/local/bin`, so `aws` isn't found. Absolute paths + redirected output (`>> /var/log/backup.log 2>&1`) surface and fix it.

19 Real-Life Analogies

One office building with a facilities team — every Linux concept is a room, a key, or a routine.

Linux concept	Real-life analogy
Kernel / userland split	The building's locked utility core (power, water, security) vs the tenant offices. Tenants can't touch the wiring directly; they file a request at the service desk (syscall), and facilities does the privileged work.
”Everything is a file”	Every room, vending machine, and notice board has the same kind of labelled door you open the same way — so one master key (the file API) works on all of them.
Inode vs filename	The filename is a label on a mailbox; the inode is the actual mailbox with the mail inside. Peel off the label and stick it on another mailbox (hard link) — same mail, two labels.
rm = unlink	Removing a label from a mailbox. The mail stays until the *last* label is gone **and** no one is still standing there reading it (open fd).
Deleted-but-open file	You peel off the only label while the night-shift clerk is mid-letter, reading from that mailbox. The mailbox (and its space) stays reserved until the clerk finishes and walks away.
Hard vs soft link	Hard link: a second label on the same mailbox. Soft link: a sticky note saying ”see mailbox 42 down the hall” — useless if mailbox 42 is later removed.
Permissions (rwx)	Door access tiers: the owner has a full key, their team has a swipe card, everyone else can only read the nameplate. `chmod 777` props every door open with a doorstop.
setuid (passwd)	A locked records room only HR may enter. You can't go in, but there's a sanctioned intercom (a setuid tool) that lets you make *one specific* change (”reset my badge PIN”) under HR's authority, nothing more.
Sticky bit (/tmp)	A shared coat closet anyone may hang coats in, but a rule that you can only take *your own* coat — not your neighbor's.
Process / fork / exec	Hiring: `fork` clones an existing employee with all their context; `exec` then hands that clone a completely new job description and they become a different worker (same badge number/PID).
Zombie process	An employee who has left, but HR never filed the exit paperwork — their badge still sits in the system roster doing nothing, cluttering it.
Orphan process	An employee whose manager quit; HR (PID 1) automatically reassigns them and will file their paperwork properly when they leave.
D-state (uninterruptible sleep)	A worker who walked into the bank vault (kernel I/O) and the time-locked door shut. You can shout ”you're fired!” (SIGKILL) all you want — they literally cannot hear or respond until the vault opens.
SIGTERM vs SIGKILL	”Please wrap up and head home” (grab your coat, save your work) versus security physically carrying you out mid-sentence, papers scattering.
Pipe	An assembly line: each station does one small transform and slides the product to the next, all working simultaneously.
stdout vs stderr	Two out-trays on each desk: one for finished work that flows down the line, one for ”problems” notes routed to the supervisor — kept separate on purpose.
Load average	The number of people either working or stuck waiting at the one printer, averaged over time. Twenty people ”in the system” with the CPU idle means nineteen are jammed at the broken printer (disk), not actually working.
OOM killer	The fire marshal: when the building exceeds capacity, they forcibly remove the largest occupant to keep the whole building from collapsing — no warning, no goodbye.

Linux concept	Real-life analogy
Namespaces	Soundproofed, one-way-mirrored offices: each tenant sees only their own rooms and believes they have the whole floor.
cgroups	The metered utilities cap per office: "this tenant gets at most 0.5 kW and 512 sq ft" — exceed the power cap and you're throttled; exceed the space cap and the fire marshal evicts you.
cron	The night cleaning crew with a fixed schedule taped to the door — but they arrive to a stripped-down building (bare environment), so you must spell out every supply location (absolute paths).

20 Memory Tricks / Mnemonics

- **fd 0/1/2** = **i**n, **o**ut, **e**rror → "**0 In, 1 Out, 2 Errors**."
- **chmod octal:** r=**4**, w=**2**, x=**1** → 7=rwx, 6=rw-, 5=r-x, 4=r--. Common: **755** dirs/bins, **644** files, **600** secrets.
- **Special-bit leading digit: 4**=setuid, **2**=setgid, **1**=sticky → `4755`, `2775`, `1777`.
- **Process states "Really Sleepy Dogs Zzz Tired": R**unning, **S**leep (interruptible), **D** (uninterruptible — disk, **D**eaf to KILL), **Z**ombie, **T** (stopped).
- `D` **= Disk = Deaf** (immune to `kill -9`, waiting on I/O).
- **TERM before KILL** — "knock, then break the door." `-9` is the last resort.
- `2>&1` **must come AFTER** `>` — "redirect the *output* first, *then* glue errors onto it."
- `sort | uniq -c` — "**sort to group, uniq to count.**" uniq only sees *adjacent* dupes.
- **USE = "Uncle Sam's Equipment"** — Utilization, Saturation, Errors, per resource.
- **Slow-server sweep: "CPU, Mem, Disk, Net, Logs"** → "**C**an **M**y **D**isk **N**ot **L**ag?"
- **Load > cores isn't always bad; load counts R *and* D** → "Load = busy **or** blocked."
- **Container = "See vs Use":** namespaces = what you **See**, cgroups = what you **Use**.
- **Script safety:** `set -euo pipefail` → "**e**rrors **u**nset **pipe**: fail fast."
- `df` **vs** `du`**:** d**f** = **F**ree (filesystem view, sees deleted-open); d**u** = **U**sage (walks names).

21 Common Interview Questions

21.1 Q1: A production server is slow --- walk me through debugging it.

Model answer: First I frame the symptom: what's slow (latency? throughput?), since when, and what changed (deploy, traffic, config). Then I do a structured **USE sweep** rather than random commands. **CPU:** `uptime` (load vs `nproc`), `top`/`mpstat -P ALL 1` — check `%us` (app CPU-bound), `%sy` (syscall-heavy), `%wa` (I/O wait, *not* CPU), `%st` (VM steal). **Memory:** `free -h` and `vmstat 1` — `si`/`so` nonzero means active swapping; `dmesg` for OOM kills. **Disk:** `iostat -xz 1` for `%util`/`await`, `iotop` for the culprit process. **Network:** `ss -s`, `sar -n DEV 1`, retransmits via `netstat -s`. Then I drill into the hottest process with `ps --sort=-%cpu`, `strace -c -p <pid>`, and finally the logs (`journalctl -u svc`, `dmesg`). The key reasoning: high load with idle CPU means I/O wait or D-state processes, and a full disk masquerades as many problems — so I check those early.

Follow-ups:

- *Load is 30 but CPU is 90% idle — what's happening?* → Load counts R **and** D (uninterruptible sleep). Processes are stuck in `D` on disk/NFS I/O; check `iostat` and list D-state procs. `kill -9` won't help.
- *How do you find which process is using the disk?* → `iotop`, or `pidstat -d 1`.
- *What if it only happens under load?* → Resource saturation: thread/connection pool, fd limits

(`ulimit -n`), cgroup throttling. Load-test at realistic concurrency.

21.2 Q2: SIGTERM vs SIGKILL --- and why does it matter for deploys?

Model answer: SIGTERM (15) is a polite, *catchable* "please shut down." A process can install a handler to flush buffers, finish in-flight requests, close DB connections, deregister from the load balancer, then exit cleanly. **SIGKILL (9)** is *uncatchable and unblockable* — the kernel destroys the process with no chance to clean up, risking data loss or corruption. The correct sequence (what `systemctl stop`, `docker stop`, and Kubernetes all do) is: send SIGTERM, wait a grace period, then SIGKILL only if it's still alive. This is the foundation of **zero-downtime deploys** — a service that traps SIGTERM and drains gracefully drops no connections; one that ignores it gets KILLed mid-request.

Follow-ups:

- *Which signals can't be caught?* → SIGKILL (9) and SIGSTOP (19).
- *How do you implement graceful shutdown in a script?* → `trap cleanup SIGTERM SIGINT`.
- *You sent SIGKILL and the process won't die — why?* → It's in `D` state (uninterruptible sleep on I/O); the signal can't be delivered until the I/O completes. Fix the I/O.

21.3 Q3: The disk is 100% full but `du` shows almost nothing. Why?

Model answer: Two classic causes. **(1) Deleted-but-open file:** a process (say a logger) still holds an open fd to a file that was `rm`'d. `rm` only *unlinks* the name; because the inode still has an open fd, its data blocks stay allocated. `df` (filesystem counters) sees the space; `du`/`ls` (which walk directory names) don't, because the name is gone. Find it with `lsof | grep deleted`, then truncate the fd (`: > /proc/<pid>/fd/N`) or signal/restart the process to reopen — *not* another `rm`. **(2) Inode exhaustion:** millions of tiny files used up all inodes; `df -h` shows free bytes but writes fail with `No space left on device`. `df -i` reveals it. A third possibility is a large file hidden under a mount point.

Follow-ups:

- *Why doesn't another `rm` help?* → The name is already gone; the fd is what's holding the space.
- *How do you prevent the logger case?* → logrotate sends SIGHUP so the daemon reopens its log instead of writing to the deleted inode.

21.4 Q4: Explain Unix permissions, octal, and the special bits.

Model answer: Each file has an owner, a group, and rwx bits for owner/group/other, shown as `rwxr-xr-x`. In octal r=4, w=2, x=1: **755** = owner rwx, group/other r-x (dirs, executables); **644** = owner rw, world read (regular files); **600** = owner-only (secrets like SSH keys). On *directories*, the bits mean differently: `r`=list names, `w`=create/delete entries, `x`=enter/traverse. The three **special bits**: **setuid** (run as the file's owner — e.g. `/usr/bin/passwd` runs as root so users can update root-owned `/etc/shadow`), **setgid** (new files inherit the directory's group — shared team folders), and the **sticky bit** (on `/tmp`: world-writable but you can only delete your *own* files). `umask` subtracts default bits at creation. `chmod 777` is dangerous because it lets anyone write and execute — the fix is correct ownership plus least privilege.

Follow-ups:

- *Why can't a normal user write to `/etc/shadow` but `passwd` can?* → `passwd` is setuid-root.
- *You have `x` but not `r` on a directory — what can you do?* → `cd` in and open a file *if you know its exact name*, but not `ls` it.

21.5 Q5: What do `2>&1`, `>`, `>>`, and pipes do --- and why does order matter?

Model answer: fd 0/1/2 are stdin/stdout/stderr. `> file` redirects stdout (overwriting), `>>` appends, `2> file` redirects stderr, and `2>&1` makes fd 2 point to *wherever fd 1 currently points*. Order

matters: `cmd > file 2>&1` sends stdout to the file, then duplicates stderr onto the same target — both land in the file. But `cmd 2>&1 > file` first points stderr at the *terminal* (where fd 1 still is), *then* redirects stdout to the file — so stderr keeps going to the terminal. A **pipe** wires one command's stdout to the next's stdin via a kernel buffer, and stages run concurrently — that's the heart of the Unix compose-small-tools philosophy.

Follow-ups:

- *How do you discard all output?* → `cmd > /dev/null 2>&1` (or `&>/dev/null`).
- *How do you see output live and save it?* → `cmd | tee file`.

21.6 Q6: Write a one-liner for the top 10 IPs in an access log, and explain it.

Model answer: `awk '{print $1}' access.log | sort | uniq -c | sort -rn | head` — `awk` extracts the first field (the IP) from each line; `sort` groups identical IPs adjacently (required because `uniq` only collapses *adjacent* duplicates); `uniq -c` collapses each group and prefixes the count; `sort -rn` sorts numerically descending by that count; `head` takes the top 10. The `sort | uniq -c | sort -rn` idiom answers any "most common X" question — top URLs, top error codes, top users. To filter to errors first, prepend `awk '$9 ~ /^5/'` or a `grep`.

Follow-ups:

- *Only count 500s?* → `awk '$9==500 {print $1}' …`.
- *Requests per minute?* → extract the timestamp substring and `uniq -c`.
- *Follow it live?* → `tail -f` the log into the pipeline.

21.7 Q7: What's a zombie process, and how is it different from an orphan?

Model answer: A **zombie** is a child that has *exited* but whose exit status the parent hasn't collected via `wait()`. It holds only a process-table slot (shows as `Z`/`<defunct>`). It's harmless individually but signals a parent bug — the parent isn't reaping, usually by failing to handle `SIGCHLD`. You can't kill a zombie (it's already dead); you fix or signal the *parent* to reap, or the parent's death re-parents it. An **orphan** is the opposite: the *parent* died while the child lives. The child is re-parented to PID 1 (`systemd`/`init`), which dutifully reaps it — so orphans are harmless. In containers, a naive PID 1 that doesn't reap leaves zombies piling up; the fix is a proper init like `tini`.

Follow-ups:

- *How do you prevent zombies?* → Parent handles `SIGCHLD` and `wait()`s; or run `tini`/`--init` as PID 1.
- *Can a zombie consume real resources?* → Only a PID slot, but enough of them exhaust the PID space.

21.8 Q8: How do containers isolate processes? What actually is a container?

Model answer: A container is just a normal Linux process (tree) wrapped in two kernel features. **Namespaces** isolate what it can *see* — PID (it's PID 1, can't see host processes), NET (own interfaces, IPs, ports), MNT (own root filesystem), UTS (own hostname), IPC, and USER (root inside maps to unprivileged outside). **cgroups** limit what it can *use* — `memory.max`, `cpu.max`, I/O, PID count. There's no guest kernel — every container shares the host kernel, which is why they start instantly and are lighter than VMs. Docker just orchestrates: create namespaces, set up a cgroup, `pivot_root` into the image's overlayfs layers, drop capabilities, and exec the entrypoint. You can build one by hand with `unshare`.

Follow-ups:

- *What's `OOMKilled` in Kubernetes?* → The container hit its cgroup `memory.max`; the cgroup OOM killer SIGKILLed it (host RAM was fine).

- *Container latency spikes but host CPU is idle?* → CPU **throttling** at `cpu.max`; check `nr_throttled` in `cpu.stat`.
- *VM vs container?* → VM virtualizes hardware with a full guest kernel (stronger isolation, heavier); a container shares the host kernel (lighter, weaker isolation).

21.9 Q9: A process is hung. How do you find out what it's doing?

Model answer: First `ps -o state,wchan -p <pid>` — the state tells me a lot: `S` waiting on an event, `D` stuck in uninterruptible I/O, `R` actually running (possibly busy-looping). Then `strace -p <pid>` to attach and see the exact syscall it's blocked in: a stuck `read`/`recvfrom` means it's waiting on a slow or dead peer; a `futex(... WAIT)` means lock contention or deadlock; *no output at all* means it isn't making syscalls — it's spinning in userspace, so I switch to `perf top -p <pid>`. `lsof -p <pid>` (or `/proc/<pid>/fd`) resolves which file/socket an fd refers to, so I learn *what* it's waiting on. For threaded apps I dump all thread stacks (`/proc/<pid>/task/*/stack`, or `jstack` for the JVM).

Follow-ups:

- *strace shows nothing — what now?* → It's CPU-bound in userspace; profile with `perf`.
- *It's stuck in `D` — can you kill it?* → No; SIGKILL can't be delivered until the I/O completes. Fix the underlying I/O.

21.10 Q10: What does load average mean, and is load 8 on an 8-core box a problem?

Model answer: Load average is the number of processes in state **R (runnable)** or **D (uninterruptible sleep)**, averaged over 1/5/15 minutes — so it's *not* a pure CPU metric. Load 8 on 8 cores means, on average, exactly as many runnable/blocked tasks as cores — roughly fully utilized but not necessarily *over*loaded; whether it's a problem depends on whether those are CPU-runnable (work getting done) or D-state (blocked on a slow disk, doing nothing). The critical insight: a box can show load 30 with the CPU 90% idle because 30 processes are stuck in `D` waiting on failing storage. So I always compare load to `nproc` *and* check `%iowait`/D-state before concluding it's a CPU problem.

Follow-ups:

- *How do you tell CPU-bound from I/O-bound load?* → `mpstat`/`top` `%us` vs `%wa`; list D-state processes.
- *Why are there three numbers?* → 1/5/15-minute averages — rising 1-min above 15-min means the load is *growing*.

21.11 Q11: How do you make a script reliable and safe?

Model answer: Start with `set -euo pipefail`: `-e` exits on any error (don't barrel past failures), `-u` errors on unset variables (catches typos that would otherwise expand to empty — turning `rm -rf "$DIR/"` into `rm -rf /`), and `pipefail` makes a pipeline fail if any stage fails. **Always quote variables** (`"$var"`) to prevent word-splitting and glob disasters on filenames with spaces. Use `trap 'cleanup' EXIT` for guaranteed cleanup (temp files) and `trap … TERM INT` for graceful interruption. Check exit codes with `$?` and chain logic with `&&`/`||`. Use `mktemp` for temp files, `local` in functions, and log to stderr so stdout stays usable in a pipeline.

Follow-ups:

- *Why quote variables?* → Unquoted `$file` with a space becomes two arguments — `rm $file` could delete the wrong things.
- *What does `trap … EXIT` give you?* → Cleanup runs no matter how the script exits (success, error, or signal).

21.12 Q12: A service won't start --- "bind: address already in use." How do you fix it?

Model answer: Something is already bound to the port. `ss -tlnp | grep :<port>` shows the owning PID and program. If it's a stale instance of my own service, I stop it gracefully

(SIGTERM) and restart. If it's a lingering `TIME_WAIT` socket from a previous crash, I either enable `SO_REUSEADDR` in the app (so it can rebind immediately) or wait the TIME_WAIT out. A related gotcha: if the service starts but remote clients get "connection refused," check that it bound to `0.0.0.0` (all interfaces) rather than `127.0.0.1` (localhost only) — `ss -tlnp` shows the bind address.

Follow-ups:

- *Difference between "connection refused" and "connection timed out"?* → Refused = reached the host but nothing's listening (or firewall RST); timed out = packets dropped silently (firewall DROP, host down, routing).
- *How to test a port without the app?* → `nc -zv host port`.

22 Senior-Level Discussion Points

- **"Everything is a file" is an architecture, not a slogan.** Because devices, sockets, pipes, and kernel state all share the file API, *one* set of tools (`cat`, redirection, `lsof`, `epoll`) works across all of them, and observability comes nearly for free via `/proc` and `/sys`. This uniformity is why the Unix model has survived 50 years.
- **Graceful shutdown is a correctness property, not a nicety.** For stateful services, ignoring SIGTERM means SIGKILL mid-write → torn data, lost in-flight requests, connection resets on every deploy. Senior engineers wire SIGTERM → drain → flush → exit, and set the orchestrator's grace period to match the real drain time.
- **Containers demystified prevent whole classes of incidents.** Knowing that `OOMKilled` = cgroup `memory.max` (not host RAM) and that latency-with-idle-CPU = `cpu.max` throttling lets you fix the *right* knob. Knowing PID 1 doesn't reap by default explains zombie pileups and stuck shutdowns.
- **Load average is a trap for the unwary.** It conflates CPU-runnable and I/O-blocked (D-state) tasks. Reasoning about it correctly — compare to core count, then split CPU vs I/O wait — separates people who've operated systems from people who memorized that "high load = bad."
- **Least privilege end to end.** Services run as dedicated unprivileged users, secrets are `600`, capabilities are dropped in containers, setuid binaries are minimized and audited, and `sudo`/`chmod 777` are smells. Security posture is largely a permissions-and-ownership discipline.
- **The kernel/userland boundary has a cost.** Syscalls are mode switches (~hundreds of ns); chatty syscall patterns dominate `%sy` time. `strace -c` quantifies it, and the fix is batching (bigger reads/writes, `sendfile`, `io_uring`) — directly connecting CLI debugging to performance engineering (§09).
- **`/proc` and `/sys` are the live source of truth.** When tools disagree or aren't installed, `/proc/<pid>/{status,fd,stack,maps}`, `/proc/loadavg`, `/proc/meminfo`, and the cgroup files under `/sys/fs/cgroup` give you ground truth straight from the kernel.

23 Typical Mistakes Candidates Make

- **Reaching for `kill -9` first.** Always SIGTERM (graceful) before SIGKILL. `-9` mid-write corrupts data and skips cleanup; it's the last resort, not the default.
- **`chmod 777` as a "fix."** A security hole. The real fix is correct *ownership* + least-privilege modes (`chown` the service user, `755`/`600`).
- **Forgetting to check the disk.** A full filesystem (or inode exhaustion) masquerades as a

dozen unrelated failures. `df -h` and `df -i` belong near the start of any triage.

- **Not knowing the deleted-but-open-file trap.** Trying another `rm` when `df` is full but `du` is empty, instead of `lsof | grep deleted`.
- **Confusing stdout and stderr**, or getting `2>&1` ordering wrong (`2>&1 > file` doesn't capture stderr to the file).
- **Misreading load average** as a pure CPU metric, ignoring D-state/I/O wait.
- **`sort | uniq -c` without the `sort`** — `uniq` only collapses adjacent duplicates, so unsorted input gives wrong counts.
- **Unquoted shell variables** → word-splitting/glob disasters on filenames with spaces; and no `set -euo pipefail`, so scripts silently barrel past errors.
- **`kill -9` on a `D`-state process** and being surprised it doesn't die — the signal can't be delivered until the I/O completes.
- **Treating a container as a VM** — not realizing `OOMKilled`/throttling come from cgroup limits, and that there's no guest kernel.
- **Running cron jobs assuming a login shell** — absolute paths and explicit env are required; bare `PATH` breaks "works by hand" scripts.
- **Editing files as root with no backup**, or `sed -i` without testing first / `-i.bak`.

24 How This Connects to Other Topics

Topic	Connection
Operating Systems (§02)	The theory beneath these commands: process states & `fork`/CoW, CFS scheduling (`nice`), virtual memory & page cache (`free`/`vmstat`), inodes & journaling, syscalls & the privilege rings.
Performance Engineering (§09)	USE method, load average, flame graphs (`perf`), bottleneck isolation (CPU vs I/O vs net), GC-vs-swap interactions — this file is the *tooling* layer for that methodology.
Networking (§03)	`ip`/`ss`/`dig`/`curl`/`tcpdump` map to TCP/IP, DNS, TLS, sockets; `ss` states (TIME_WAIT, SYN-RECV) reflect the TCP state machine.
Cloud / Containers (§04)	Docker/Kubernetes = namespaces + cgroups + overlayfs; `OOMKilled`, CPU throttling, and limits are cgroup mechanics.
Observability (§18)	`journalctl`, structured logs, `/proc` metrics, and `dmesg` are the raw inputs that feed Prometheus/Grafana/Datadog dashboards.
Security (§19)	Permissions, setuid, least privilege, capabilities, and user namespaces are the host-level security primitives.
System Design	"Deploy", "scale", "cache in memory", "graceful restart" all bottom out in Linux primitives — page cache, cgroup limits, SIGTERM handling, fd/connection limits.

25 FAANG Interview Tips

1. **Narrate a structured sweep** for "debug the slow server" — CPU → mem → disk → net → process → logs (USE method). The *method* impresses more than any single command.
2. **Name the right probe for the symptom:** `lsof` for deleted-open files and fds, `iostat`/D-state for I/O-bound load, `dmesg` for OOM/hardware, `ss` for port conflicts, `strace` for hangs, `perf` for userspace spins.
3. **Volunteer the gotchas** — deleted-but-open file, load counts D-state, `kill -9` can't touch `D`,

bound-to-127.0.0.1, cron's bare environment. These signal real operational scars.
4. **Always SIGTERM before SIGKILL**, and tie it to zero-downtime deploys and data integrity.
5. **Mention least privilege** (`chown` + `755`/`600`, no `chmod 777`, run as a service user) whenever permissions come up.
6. **Write the log one-liner without hesitating:** `… | sort | uniq -c | sort -rn | head`. Explain *why* the `sort` before `uniq -c` is necessary.
7. **Demystify containers** as namespaces (see) + cgroups (use), no guest kernel — and connect `OOMKilled`/throttling to cgroup limits.
8. **Tie commands back to OS theory** — `%wa` = I/O wait, swapping = page cache pressure, `nice` = CFS weight — to show depth, not memorization.
9. **Start scripts with `set -euo pipefail` and quote variables** when asked to write shell — it signals you've been burned before.
10. **Use `/proc` and `/sys`** as the ground-truth fallback when a tool is missing or two tools disagree.

26 Revision Cheat Sheet

26.1 10-Minute Summary

Linux is a kernel + GNU userland; the **shell** composes small text tools with **pipes** (Unix philosophy). Two anchors: **"everything is a file"** (devices, sockets, `/proc` all share the file API) and the **kernel/userland split** (programs do privileged work only via **syscalls**).

A **filename points to an inode**, not the file. `rm` *unlinks* (drops a name, decrements link count); data dies only when no name **and** no open fd remain — hence the **deleted-but-open file** keeping a disk 100% full (`lsof | grep deleted`). `df` (filesystem view) and `du` (walks names) disagree exactly there, and `df -i` catches inode exhaustion.

Permissions: octal r=4/w=2/x=1 → **755** dirs/bins, **644** files, **600** secrets; special bits **setuid** (`passwd` runs as root), **setgid** (inherit dir group), **sticky** (`/tmp`). `chmod 777` is a hole; fix with ownership + least privilege.

Processes: `fork+exec+wait`; states **R/S/D/Z/T** — `D` = uninterruptible I/O sleep (immune to `kill -9`), `Z` = zombie (parent didn't reap). **Signals: SIGTERM** (15, catchable, graceful) before **SIGKILL** (9, uncatchable) — the basis of zero-downtime deploys; `trap` handles them in scripts.

Redirection: fd 0/1/2; `>`/`>>`/`2>`/`2>&1` (order matters); pipes stream concurrently. **Text:** `grep`/`sed`/`awk` + `… | sort | uniq -c | sort -rn | head` for "most common X."

Performance: USE per resource. **Load average** counts R **and** D — high load + idle CPU = I/O wait. `vmstat`/`mpstat` (CPU), `free`/`vmstat si/so` (memory/swap), `iostat -x` (disk await/%util), `ss`/`sar` (net), `lsof`/`strace` (what a process touches), `dmesg` (OOM).

Containers = namespaces (see) + cgroups (use), no guest kernel; `OOMKilled` = cgroup `memory.max`, latency-with-idle-CPU = `cpu.max` throttling. **systemd** manages services (`systemctl`, `journalctl -u -f`); **scripts** start with `set -euo pipefail` and quote variables.

26.2 Cheat Sheet Table

Concept	One-liner
fd 0/1/2	stdin / stdout / stderr
Pipe |	left.stdout → right.stdin, concurrent
Redirect	`>` overwrite, `>>` append, `2>` errors, `2>&1` merge (after `>`)
chmod octal	r=4 w=2 x=1 → 755 dirs, 644 files, 600 secrets
Special bits	4=setuid (passwd), 2=setgid (group inherit), 1=sticky (/tmp)
inode / rm	name → inode; `rm` = unlink; data freed at link=0 AND no open fd

Concept	One-liner
deleted-open file	`df` full, `du` empty → `lsof | grep deleted`
df vs du	df=filesystem free (sees deleted-open); du=walks names; `df -i`=inodes
Hard vs soft link	hard = 2nd name, same inode; soft = path pointer, can dangle
States R/S/D/Z/T	run / sleep / **uninterruptible I/O** / zombie / stopped
D-state	stuck on disk/NFS; `kill -9` can't touch it; fix the I/O
Zombie vs orphan	zombie = unreaped child (parent bug); orphan → PID 1 adopts (fine)
SIGTERM (15)	graceful, catchable — default `kill`; drain then exit
SIGKILL (9)	forceful, uncatchable — last resort
trap	`trap cleanup EXIT TERM INT` — graceful shutdown / cleanup
Load average	count of R + D tasks; compare to `nproc`; counts I/O wait
USE method	Utilization, Saturation, Errors — per resource
CPU / mem / disk / net	mpstat,vmstat / free,vmstat si/so / iostat -x / ss,sar
lsof / strace	open files & fds / live syscalls of a process
OOM killer	out of RAM → SIGKILL a victim; `dmesg | grep -i oom`
Container	namespaces (see) + cgroups (use); no guest kernel
OOMKilled / throttle	hit cgroup memory.max / cpu.max (idle host CPU)
systemd	`systemctl start/stop/status`, `journalctl -u svc -f`
ulimit -n / swappiness	raise fd limit; lower swap eagerness for hot apps
Script safety	`set -euo pipefail`, quote `"$var"`, `trap`, `$?`
Net triage	`ss -tlnp`, `dig +short`, `curl -w`, `nc -zv host port`, `tcpdump`
Log one-liner	`awk '{print $1}' | sort | uniq -c | sort -rn | head`
Debug order	CPU → Mem → Disk → Net → Process → Logs

Golden rule: Compose small text tools with pipes; a filename is just a pointer to an inode; prefer graceful SIGTERM over `kill -9`; check the disk (and inodes) early; reason about resource saturation with USE — and remember that high load with an idle CPU means I/O wait, not a CPU problem.

Last updated: 2026-06-17 | Topic: Linux & the Command Line | Level: FAANG Backend / SRE / Infra